

Physical Design Inputs

A Deep Reference

Compiled for Kaushik Vada

May 29, 2026

How to Read This Book

This book is a reference, not a transcript. Each chapter takes one Physical Design input artifact and goes deeper than the source lecture, with worked numbers, diagrams, and tight chunking. Read it the same way you would read a textbook chapter you actually intend to remember.

The callout boxes are a language. Learn them once.

- **TL;DR (purple)** opens every chapter with the payload in 3 to 4 bullets. From Chapter 8 onward, the first bullet is a callback to the prior chapter so the spaced repetition stays warm.
- **Memory hook (yellow)** gives you a vivid, sensory analogy. The brain remembers weird better than tidy.
- **Gut check (gray)** asks a question. **Cover the gray answer with your hand and try to recall first.** The recall attempt is the point, not the answer.
- **Pause (cyan)** is a mid-chapter recap. If you can repeat the Pause back to yourself, keep reading. If not, scroll back.
- **Key Idea (blue)** is the single most important sentence of the chapter. Underline it mentally.
- **Watch Out (red)** is a common mistake. These are the things interviewers ask about.
- **Why PD Cares (orange)** translates the chapter into three lenses: Arm PD internship, generic foundry-flow PD engineer, and VLSI interview prep.
- **Sidebar (black)** is optional depth. Skip if tired. Come back when curious.
- **Checkpoint (green)** closes the chapter with a 2-second mnemonic. If you can say the mnemonic aloud, the chapter is yours.

Rule for when something is not clicking. Scroll back to the most recent Pause or Memory hook and re-read from there. The book is built so that retreating one box is always enough to recover momentum.

Rule for the Interview-ready 90-second answer at the back. Cover it. Recite your own answer out loud. Then uncover and compare. This is the testing effect; it is the cheapest retention boost available.

Contents

How to Read This Book	ii
1 Physical Design Inputs: The Big Picture	1
1.1 The six inputs at a glance	2
1.1.1 What each artifact actually encodes	2
1.2 Walking the six, one at a time	3
1.3 A back-of-envelope on input volume	3
1.4 Why the inputs are split, not merged	4
2 The Netlist: A Wire-Level Blueprint	6
2.1 What a Netlist Really Is	6
2.2 The 4-Gate Example	7
2.3 The Verilog Form	7
2.4 Technology-Independent vs. Synthesized	8
2.5 Why Cell Names Matter (Chapter 9 Foreshadow)	8
2.6 Worked Example: Counting What PD Sees	8
3 The Timing Library (.lib): How Cells Tell Time	11
3.1 The Two Axes of Cell Delay	12
3.2 What a Liberty File Actually Looks Like	13
3.3 Worked Example: Bilinear Interpolation by Hand	14
3.4 Setup, Hold, and the Other Tables	15
3.5 Why CCS Exists (and Why NLDM Quietly Lies at 7nm)	15
4 The LEF File: Physical Abstraction of Cells and Layers	18
4.1 Why the placer is blind to your beautiful layout	18
4.2 Anatomy of a MACRO block	19
4.3 Worked example: site arithmetic for an INV_X1	20
5 The Constraints File (SDC)	23
5.1 What lives inside the SDC	23
5.1.1 The timing waveform: launch, capture, slack	24
5.1.2 Anchoring the boundary: input and output delay	24
5.2 The Big 5 SDC commands, by example	25
5.2.1 create_clock and create_generated_clock	25
5.2.2 set_input_delay, set_output_delay, virtual clocks	25
5.2.3 set_clock_uncertainty, set_false_path, set_multicycle_path	25
5.3 Worked example: setup slack arithmetic	26

6	The Technology File: Where Physics Becomes Rules	29
6.1	The Stack You Are Actually Routing On	29
6.2	What Lives Inside the File	30
6.3	Worked Example: Counting Tracks Across a Channel	31
6.4	Watch Out	32
7	The DEF File: A Snapshot of Where Everything Lives	34
7.1	What DEF Actually Is	34
7.2	The Floorplan View	35
7.3	Inside the File	35
7.4	DEF Lives at Every PD Checkpoint	36
7.5	Worked Example: Coordinate Arithmetic	37
8	The CTS Spec: How You Tell the Tool to Build the Clock	40
8.1	What the CTS Spec Actually Is	40
8.2	Clock Skew Groups	41
8.3	NDR: Non-Default Routing Rules	42
8.4	Useful Skew	43
8.5	Clock Mesh vs Clock Tree	44
9	Parasitic Files: ITF, the Mapping File, and SPEF	46
9.1	ITF: The Interconnect Technology File	46
9.2	The Mapping File: Tiny, Mission Critical	47
9.3	SPEF: Where Parasitics Become a File You Read	48
9.4	How STA Actually Uses SPEF	49
9.5	SPEF Flavors and Corners	50
10	What Changes in the Real World	53
10.1	MMMC: the Liberty view explosion	53
10.1.1	Counting the Liberty views directly	53
10.1.2	How Innovus actually structures it	54
10.2	Pin access: when LEF stops being enough	55
10.2.1	Why the 28 nm pin shape fails at 7 nm	55
10.3	AOCV and POCV: same SDC, different answer	56
10.3.1	Worked example: a 12-level path at 1.0 ns nominal	56
10.4	DEF growth through the flow, with post-CTS as the surprise	57
10.4.1	Worked example: a 1M-instance block	57
10.5	Why PD Cares	58
11	Common Mistakes	59
12	Interview-Ready 90-Second Answer	60
13	Cumulative Recall Sheet	61
14	Practice Problems	63
	Companion Videos	65

Chapter 1

Physical Design Inputs: The Big Picture

TL;DR

- A Physical Design (PD) job is fed by **six artifact families**: a gate-level **netlist**, an **SDC** constraints file, a **LEF** (Library Exchange Format), one or more **.lib** timing views, a **Technology file** (often with an ITF parasitic file plus mapping), and a **DEF** (Design Exchange Format) for floorplan and ports.
- Each input encodes a different question. Drop one and a different stage of the flow goes blind: synthesis hand-off, floorplanning, timing, parasitic extraction, or DRC all rely on a specific artifact.
- Modern advanced-node designs do not consume one .lib. They consume **dozens of .lib views across PVT corners and modes** (MMMC). The same is true for LEFs and parasitic tech views at multiple metal stacks.
- This chapter is the map. Chapters 8 onward zoom into each artifact one at a time.

An interview question you cannot dodge

Hook

A fresher walks into an Arm PD interview. First question: “Tell me every file you would expect on your disk before you can run a single **floorplan** command.” Silence in the room. The candidate names two files. The interviewer waits. Six were expected. Why six, and not one giant database?

The instinct is to imagine one master file. In reality, six separate artifacts get handed to PD because each one is **authored by a different team and consumed by a different engine**. The netlist is a synthesis output. The .lib is a foundry signoff deliverable. The LEF is a library characterization output. The SDC is written by the architect or front-end lead. The tech file is foundry IP. The DEF is something PD itself iterates on. **Six origins, six update cadences, six file formats.**

Key Idea

Physical design is not a single tool ingesting a single file. It is a **pipeline of engines** (synthesis hand-off, floorplan, placement, CTS, route, signoff) where each engine demands a specific input that no other input can substitute for. Lose one, lose that engine.

1.1 The six inputs at a glance

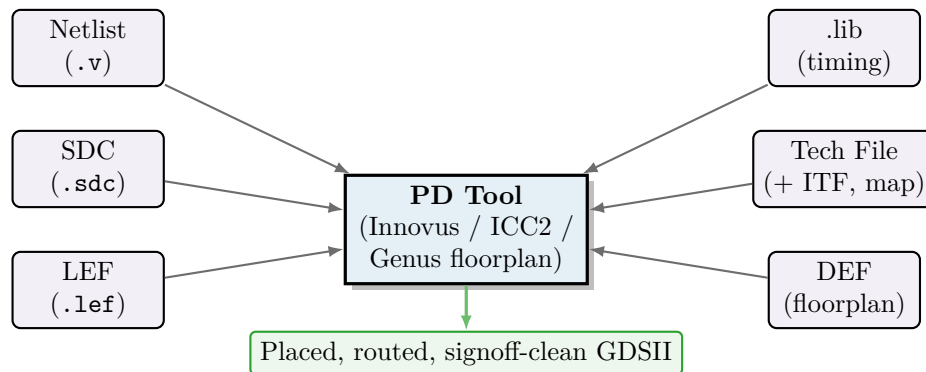


Figure 1.1: The six PD input artifacts feeding a single PD tool invocation. Authored upstream, consumed downstream, all required before signoff.

Memory hook

Think of the PD tool as a **surgeon in an operating theater who refuses to cut until six specific trays of instruments are laid out by name**. Hand the surgeon five trays and they walk out. The sixth missing tray is not a warning, it is a hard stop.

1.1.1 What each artifact actually encodes

Artifact	Encodes	Author	Consumed at
Netlist	Connectivity	Synthesis	All stages
SDC	Timing intent	Architect	STA, CTS, opt
LEF	Cell footprint	Lib team	Floorplan, place, route
.lib	Cell timing	Foundry char	Timing, opt, CTS
Tech File	Metal stack & DRC	Foundry	Route, extraction
DEF	Floorplan, IO	PD itself	Place, route, ECO

Figure 1.2: Each input answers a question no other input answers. The author column is why no single file could replace the set.

Gut check

If you delete the LEF but keep the .lib, what is the very first PD command that breaks?

Anything that needs to place a cell on the floorplan. The .lib knows the cell's timing, but it does not know the cell's shape. Placement needs width, height, pin locations, all of which live in the LEF. Timing analysis on an unplaced design might still run, but you cannot draw a single rectangle.

1.2 Walking the six, one at a time

Move 1. Netlist. A gate-level Verilog file enumerating every instance and every wire. This is the only file that says “instance u_alu/add_42 is a NAND2_X2 whose A pin connects to net n1764.” Without it, the PD tool has no design to place.

Move 2. SDC. The Synopsys Design Constraints file. It carries clock definitions (period, uncertainty, skew, latency), I/O delays, false paths, multicycle paths, max-transition, max-cap, and other DRV limits. The architect says “this block must close at 1.2 GHz with 80 ps uncertainty.” That sentence becomes an SDC line.

Move 3. LEF. A physical abstract of every standard cell and macro. Cell name, shape, size, orientation, pin name, pin layer, pin geometry, obstruction layers. The PD tool reads this once at setup and uses it for every legalization, every routing track decision.

Move 4. .lib (Liberty). The timing view. For every input pin of every cell, at every PVT corner, it lists propagation delays, output transitions, input capacitance, and constraint arcs (setup, hold, recovery, removal) as 2D lookup tables indexed by input slew and output load. Without .lib, the tool literally cannot compute a single arrival time.

Move 5. Technology file (+ ITF + mapping). Foundry IP. Resistance and capacitance per metal layer, minimum width, minimum spacing, via rules, layer colors for multi-patterning, plus the ITF (Interconnect Technology Format) parasitic deck and a mapping file that ties ITF layer names to tech-file layer names. Without correct mapping, extraction returns garbage RC, and timing signoff diverges from silicon.

Move 6. DEF. The Design Exchange Format file holds the physical state of the block itself: die size, core area, IO port locations, macro placements, blockages, sometimes power grid. Early in the flow DEF is sparse (just floorplan). Late in the flow it is dense (every cell location, every route). DEF is the artifact PD writes back out.

Pause

Pause. You now know the six artifacts, the question each one answers, and the engine that needs it. The rest of this book unpacks one artifact per chapter. If you can recite “netlist, SDC, LEF, .lib, tech, DEF” and name one engine that breaks without each, you are at the level the interviewer is testing for.

1.3 A back-of-envelope on input volume

Beginners imagine “the .lib” as one file. On a real 7 nm SoC block, the count is not one. Walk through it with substituted values, then generalize.

Move 1. Assume 5 process corners (SS, FF, TT, SF, FS). Assume 3 voltage points (nominal, low, over-drive). Assume 3 temperatures (-40 C, 25 C, 125 C). That is $5 \times 3 \times 3 = 45$ PVT permutations.

Move 2. Not every permutation is exercised; foundry signoff typically calls out a subset, say **12 corners**.

Move 3. The design uses 3 standard-cell libraries (regular Vt, high Vt, low Vt) plus 4 memory compilers. That is 7 cell families. $12 \times 7 = 84$.lib files for this block.

Move 4. Generalize: the count of .lib views consumed is approximately

$$N_{\text{lib}} \approx N_{\text{corners}} \times N_{\text{families}}.$$

Move 5. Now repeat the same math for LEF (one per library, sometimes per metal stack option) and parasitic tech files (one per RC corner, typically **cWorst**, **cBest**, **rcWorst**, **rcBest**, **typical**, so 5). The block consumes **dozens of files just to set up MMMC**.

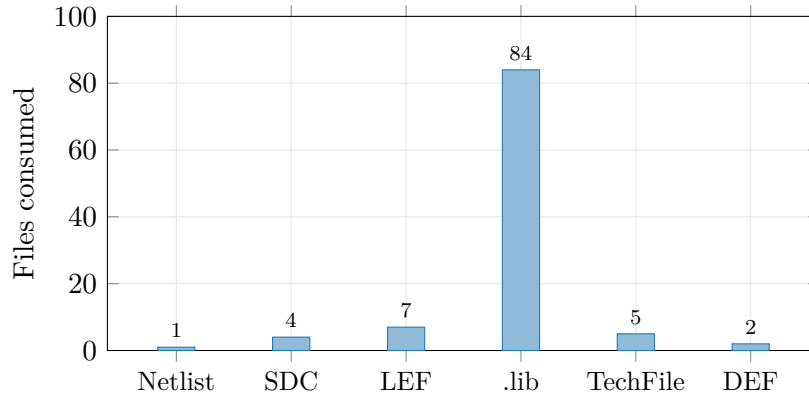


Figure 1.3: Approximate file counts a single 7 nm SoC block consumes during PD setup. The .lib bar dominates because timing must be re-evaluated at every PVT and every cell-family combination.

Gut check

On a 7 nm design with 12 corners and 7 cell families, your bring-up script complains it cannot find `ss_0p72v_125c_rvt.lib`. Which engine will fail first when you try to run anyway?

The first stage that needs that specific worst-case slow corner for hold or max-cap analysis, typically CTS or post-route timing. The placer can often limp along on a single corner, but multi-corner optimization and signoff will refuse.

1.4 Why the inputs are split, not merged

Sidebar: optional depth, skip if tired

A reasonable question: if all six artifacts feed one tool, why not merge them? The answer is **ownership and update cadence**. The foundry ships tech files once per process node revision (years). Cell libraries refresh on a quarterly basis. The architect rewrites SDC weekly during convergence. The netlist regenerates every nightly synthesis run. DEF flips on every floorplan iteration, sometimes hourly. A merged file would force the slowest-cadence team to gate the fastest-cadence team. The file boundary is a **social contract**, not a technical limit.

There is a second reason: format standards are owned by different bodies. LEF and DEF are Cadence-originated, .lib is Synopsys-originated, SDC is also Synopsys, the tech file is foundry-proprietary. The split survives because nobody can unify the standards bodies.

Watch Out

The most common fresher mistake: assuming .lib and LEF are the same thing because both contain “cell” information. They are not. **.lib is timing only, LEF is geometry only.** A cell appears in both files under the same name, but the two views never overlap in content. If you cannot answer “which file has the pin capacitance” (.lib) versus “which file has the pin location” (LEF), the interviewer will know.

Why PD Cares

- **Arm PD intern lens.** Day one, you will be handed a block bring-up script that sources a netlist, an SDC, a LEF list, a .lib list, and a tech file. You will be expected to know which environment variable points to which file and which one to grep first when bring-up fails.
- **Foundry-flow PD engineer lens.** Production flows version-pin every input. A .lib refresh from the foundry triggers a full MMMC re-characterization. The flow’s job is to detect input drift before signoff, not after.
- **VLSI interview lens.** The interviewer is not testing memorization. They are testing whether you understand that timing, geometry, intent, and parasitics are **orthogonal concerns** and that the file split mirrors the concern split. Answer in that frame and you stand out.

Checkpoint

You can now name the six PD inputs, the question each answers, the engine that breaks without each, and the rough multiplier that makes a single .lib reference into 84 actual files. The rest of the book is one chapter per artifact.

Say it in 2 seconds: *NSL-LTD: Netlist, SDC, LEF, Lib, Tech, DEF*

Chapter 2

The Netlist: A Wire-Level Blueprint

TL;DR

- **Last chapter:** PD is a pipeline of engines, each demanding one irreplaceable input. **This chapter:** we crack open the first input (the netlist) and see the graph the rest of the flow walks across.
- A **netlist** is a textual graph of the design: **cell instances** as nodes, **nets** as edges, and **top-level ports** as the boundary.
- **Synthesis** converts RTL behavior into a netlist of library cells; the same RTL can be re-synthesized into a different technology and produce a different netlist.
- Every instance name in a **synthesized (gate-level) netlist** must resolve to a cell defined in the technology `.lib`, or the PD tool will throw a missing-reference error before floorplanning even begins.

Memory hook

Hand a physical design tool a million-line netlist and it sees one thing: a giant adjacency list of cells and the wires soldering them together. Strip away the comments, the module wrappers, the friendly signal names, and what is left is a circuit graph. That graph is the only thing the placer, the router, and the timing engine ever really look at. So what does this graph actually say?

2.1 What a Netlist Really Is

A **netlist** is a connectivity description. It enumerates the **components** (cell instances) in the circuit and lists the **nodes** (nets) that those components share. A **net** is a single equipotential wire that touches two or more pins. A **port** is a wire that crosses the module boundary to the outside world.

That is the whole data structure. Three sets: instances, nets, ports. Every PD tool builds the same in-memory graph from whichever text format it was handed.

Key Idea

A netlist is a **graph of cell instances connected by nets**, with top-level ports as the only opening to the outside world; everything PD does (placement, routing, timing) operates on this graph.

2.2 The 4-Gate Example

The lecture builds a tiny majority-style circuit with three AND gates feeding one OR gate. The structural picture looks like this.

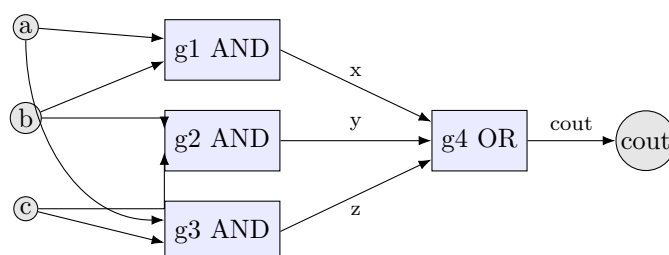


Figure 2.1: Structural view of the 4-gate example. Four instances (g1, g2, g3, g4), three internal nets (x, y, z), three input ports (a, b, c), one output port (cout).

Count what the graph contains: **4 instances**, **3 internal nets**, **4 ports**. Every wire in the picture maps to exactly one entry in the netlist text.

Memory hook

Picture a **telephone switchboard** where each operator (cell instance) holds labeled cords (pins) and every cord plugs into a single colored wire bundle (net) running across the back panel. Lose one cord-to-bundle mapping and the call drops, which in silicon means the placer refuses to start.

2.3 The Verilog Form

Here is what the four-gate circuit looks like as a technology-independent Verilog netlist.

Listing 2.1: Technology-independent gate-level netlist for the 4-gate example.

```

1 module maj_block (a, b, c, cout);
2   input a, b, c;
3   output cout;
4   wire x, y, z;
5
6   and g1 (x, a, b);
7   and g2 (y, b, c);
8   and g3 (z, a, c);
9   or g4 (cout, x, y, z);
10 endmodule

```

Read it as three blocks. The **header** declares ports. The **wire** statement names internal nets. The body **instantiates** cells, listing the output pin first and inputs after.

Gut check

In Listing 2.1, how many **nets** does the design contain in total (count ports as nets too, since each is a wire crossing the boundary)?

Seven: three input ports (a, b, c), one output port (cout), and three internal wires (x, y, z).

2.4 Technology-Independent vs. Synthesized

The lecture draws a sharp line at the end. The Verilog above uses generic **and** and **or** primitives. No PD tool can place those, because they have no area, no pin locations, no drive strength. That version is a **technology-independent netlist**: pure structure.

A **synthesized (gate-level) netlist** replaces every primitive with a **reference** to a real library cell, like AND2_X1 or OR3_X2 from a foundry standard-cell library.

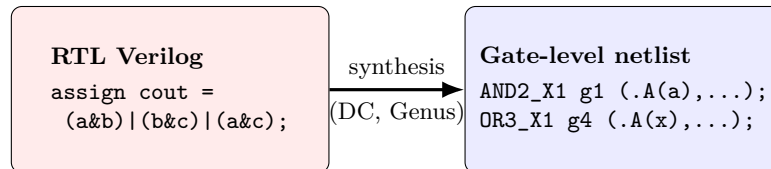


Figure 2.2: Before/after: the same intent expressed as behavioral RTL (left) and as a synthesized gate-level netlist bound to library cells (right). PD only consumes the right-hand form.

The mapping from RTL to gate-level is what the synthesis tool (Design Compiler, Genus, Yosys) does. Same RTL, different `.lib` target, different netlist. That is why one Verilog source survives a node shrink while the netlist gets thrown out and regenerated.

Memory hook

A **recipe** (RTL) survives moving to a new **kitchen** (foundry), but the **ingredient list with brand names** (netlist) gets rewritten every time, and serving the wrong brand (a cell not in the `.lib`) ruins the whole dish (the PD database refuses to load).

Pause

Pause. Before reading on, sketch what would happen if your synthesis target was a 7 nm `.lib` but you handed PD the netlist synthesized against the 28 nm library. Where in the flow does the failure show up, and what does the error message complain about?

2.5 Why Cell Names Matter (Chapter 9 Foreshadow)

Every reference name in the synthesized netlist (e.g., NAND2_X2) is a string lookup into the technology `.lib`. If the netlist says NAND2_X2 but the library only defines NAND2X2 (no underscore), the PD tool reports an unresolved reference and refuses to elaborate the design.

This is why you cannot mix-and-match. The netlist, the `.lib`, and the LEF (Chapter 9) must all agree on cell names down to the character. We will see in the next chapter why the `.lib` is the source of truth for those names.

2.6 Worked Example: Counting What PD Sees

Problem. For the netlist in Listing 2.1, compute the instance count, the net count, and the average net degree (pins per net).

Move 1. Instances. $g1, g2, g3, g4 \Rightarrow N_{\text{inst}} = 4$.

Move 2. Nets. Internal: $\{x, y, z\}$. Port nets: $\{a, b, c, \text{cout}\}$. Total $N_{\text{net}} = 7$.

Move 3. Pin count per net. Each input port drives certain gates:

- a: 1 port + g1.A + g3.A $\Rightarrow$ 3 pins
- b: 1 port + g1.B + g2.A $\Rightarrow$ 3 pins
- c: 1 port + g2.B + g3.B $\Rightarrow$ 3 pins
- x: g1.Z + g4.A $\Rightarrow$ 2 pins
- y: g2.Z + g4.B $\Rightarrow$ 2 pins
- z: g3.Z + g4.C $\Rightarrow$ 2 pins
- cout: g4.Z + 1 port $\Rightarrow$ 2 pins

Move 4. Average degree. Total pins = 3 + 3 + 3 + 2 + 2 + 2 + 2 = 17. Average pins/net = $17/7 \approx 2.43$.

Move 5. Scaling intuition. Real SoCs run roughly 1 net per instance. For a 1 M-instance block, expect on the order of **1 M nets**. Each one becomes a routing problem.

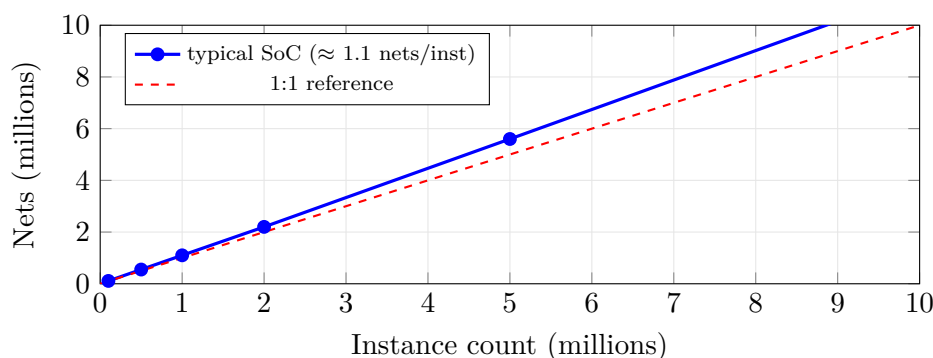


Figure 2.3: Net count vs. instance count for synthesized blocks. The slope sits slightly above 1 because high-fanout nets (clock, reset, scan-enable) push the average up.

Gut check

For a block with 250,000 instances, what is a reasonable first-cut estimate of the net count, and what single signal probably skews that number upward?

*Around 275,000 nets (1.1× rule). The **clock net** typically touches every flop, but it gets split into a tree by CTS so the raw netlist still shows it as one giant net.*

Watch Out

Common netlist misconceptions.

- **Behavioral vs. structural.** A file with `always` blocks or `assign` expressions is *not* a netlist PD can consume. PD wants pure structural Verilog: instances and connections only.
- **Blackboxes.** An instance whose module body is not in the netlist (memories, hard macros, IP blocks) is a **blackbox**. PD still needs its physical view (LEF) and timing view (.lib); otherwise it cannot place or time around it.
- **Cell name mismatch.** INVX1 in the netlist will not bind to INV_X1 in the .lib.

Spelling, case, and underscores all matter.

- **Non-uniquified hierarchical netlists.** If the same module is instantiated under two parents that need different constraints (say, one on a critical 1 GHz datapath, the other in scan-only logic), synthesis must **uniquify** by renaming each instance to a distinct module. A non-uniquified netlist applies one set of timing budgets to both, and the failing endpoint shows up only at signoff, after a week of PD work.
- **Physical-only cells without .lib entries.** Tap cells, end-caps, decaps, and ECO fillers live in the netlist (or get inserted post-synthesis) but have no timing arcs in .lib. PD tools handle this gracefully; STA-only flows can throw confusing missing-pin errors if the engineer forgot to mark them as physical-only.

Sidebar: optional depth, skip if tired

Hierarchical vs. flat netlists. A hierarchical netlist preserves module boundaries (`cpu/alu/adder/g1`); a flat netlist dissolves them so every leaf instance lives at the top (`cpu_alu_adder_g1`). Hierarchical is easier for humans and faster to elaborate; flat gives the optimizer freedom to retime and resize across former boundaries. Most modern flows stay hierarchical at top and flatten selectively inside each block.

EDIF. Older flows and some FPGA tools exchange netlists in **EDIF** (Electronic Design Interchange Format), a Lisp-like S-expression syntax. The information is identical; the grammar is just uglier. ASIC PD today is overwhelmingly Verilog.

Why PD Cares

Why PD cares.

- **ARM PD internship lens:** the first thing you do on day one is load a synthesized .v into Innovus or ICC2 and check that every reference resolves against the loaded .lib/LEF; unresolved references are the most common “my flow won’t start” ticket.
- **Generic foundry-flow PD engineer lens:** the netlist drives **instance count budgeting** (placement density), **net count budgeting** (routing congestion), and **port count budgeting** (pin planning at block boundaries); all three pop out of a 30-second `report_design` run.
- **VLSI interview lens:** expect “what is the difference between a behavioral and a structural netlist?” and “which inputs does PD need before placement?”; the netlist is always one of the three answers (netlist, .lib, LEF).

Checkpoint

You should now be able to: (1) define a netlist as an instance/net/port graph, (2) distinguish a technology-independent netlist from a synthesized one, (3) read a small Verilog gate-level netlist and count instances and nets, and (4) explain why every reference in the netlist must match a cell in the .lib.

Say it in 2 seconds: **INP: Instances, Nets, Ports**

Chapter 3

The Timing Library (.lib): How Cells Tell Time

TL;DR

- **Last chapter:** a netlist is just a graph of instances, nets, and ports. **This chapter:** we feed that graph to the timing engine, which only knows what each cell does because the .lib tells it.
- A .lib is the Liberty-format file that tells every EDA tool how each standard cell behaves: its pins, function, timing arcs, power, and leakage.
- Timing inside a .lib is stored as **NLDM lookup tables** indexed by **input slew** and **output load**; the tool reads the table and **bilinearly interpolates** when your operating point falls between grid samples.
- **CCS** and **ECSM** add a third axis (time-varying current) to model Miller capacitance accurately at sub-130 nm nodes, at the cost of much larger library files.
- Foundries deliver one .lib per **PVT corner** (slow-slow worst case, typical, fast-fast best case); a real flow loads many of them under **MMMC** so STA can sign off across all corners simultaneously.
- Tools compile .lib into a binary .db (Synopsys) for fast ingestion, but engineers always debug against the human-readable .lib.

Why this chapter exists

You hand the place-and-route tool a netlist that says `INVX1 U7 (.A(n42), .Y(n43));`. The tool stares back and asks a question you cannot answer from the netlist alone: *how long does it take for a transition on n42 to appear on n43?* That number is not in the Verilog. It is not in the library exchange format. It lives in exactly one place, and that place is the timing library. Without it, no static timing analysis runs, no gate sizing happens, no clock tree gets built, and no chip taped out.

This chapter unpacks what is actually inside that file. We will look at a real NLDM lookup table, watch a tool interpolate a delay value step by step, and then see where NLDM breaks down and CCS takes over.

3.1 The Two Axes of Cell Delay

A standard cell's delay is not a constant. It depends on **how sharp the incoming edge is** (input slew, or `tran`) and **how much capacitance the output is driving** (output load, or `cap`). A slow input edge stretches the delay; a heavy output load stretches it more. Every other gate-level timing question reduces to those two knobs.

The Liberty file captures this by storing a 2D **lookup table** per timing arc per cell. Rows are discrete input-slew values. Columns are discrete output-load values. Each cell holds a delay (and a separate table holds output slew). When the tool needs a point that is not on the grid, it interpolates.

Figure 1: NLDM delay surface for an INVX1 cell

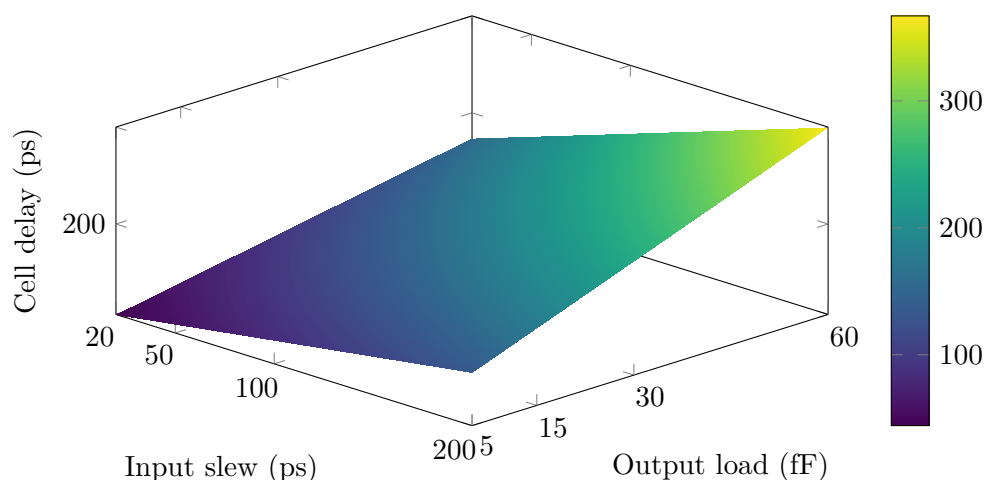


Figure 3.1: NLDM delay surface for a representative INVX1 cell. Delay grows roughly linearly with each axis and faster along the diagonal because the slew and load effects multiply. Real .lib tables sample this surface at **3 to 7 grid points per axis**.

Memory hook

Hook 1. The fork pierces the steak slowly when the steak is cold. A cold steak is a heavy load; a dull fork is a slow slew. Both make the bite take longer. Stakes: order the wrong cell drive strength and your hold-time margin evaporates before you can re-spin.

Memory hook

Hook 2. The librarian stamps every book at three temperatures. The .lib is the stamp catalog; the librarian is the foundry; the temperatures are PVT corners. Skip a corner and a chip ships that fails in the Arizona heat. Stakes: an entire wafer lot scrapped.

Gut check

Gut check 1. If you double the output load but keep input slew fixed, what happens to cell delay in an NLDM model?

It grows, but not linearly in general. The lookup table captures whatever curvature the SPICE characterization measured. For an INVX1 on a typical 28nm node, doubling load from 15fF to 30fF might raise delay by roughly 60 to 80 percent.

Gut check

Gut check 2. Why does the .lib store *two* tables (rise and fall) per timing arc instead of one?

NMOS and PMOS have different mobilities. The pull-down path through NMOS is usually faster than the pull-up through PMOS, so rise and fall delays differ. The tool needs both to compute setup and hold against the correct edge.

Pause

So far: a .lib is the file that tells the tool how a cell delays a signal, the delay depends on input slew and output load, and the data is stored as a 2D table per arc per cell. Next: we read a real Liberty snippet and then do the arithmetic the tool does.

Key Idea

A timing library is a **characterized lookup of cell behavior across PVT corners**. The NLDM model boils every cell's delay down to a 2D table indexed by input slew and output load, and the tool interpolates between grid points whenever your operating condition falls in between.

3.2 What a Liberty File Actually Looks Like

Figure 2: A real Liberty timing block for an inverter

Listing 3.1: Excerpt of an INVX1 cell in Liberty format. The `lu_table_template` (defined once at the top of the .lib) declares the index variables; each `cell_rise` / `cell_fall` group fills in the delay surface for one timing arc.

```

1 library (tsmc28_tt_0p9v_25c) {
2   time_unit : "1ns";
3   capacitive_load_unit (1, ff);
4   voltage_unit : "1V";
5   nom_voltage : 0.90;
6   nom_temperature : 25.0;
7
8   lu_table_template (delay_template_3x3) {
9     variable_1 : input_net_transition;
10    variable_2 : total_output_net_capacitance;
11    index_1 ("20.0, 100.0, 200.0"); /* ps */
12    index_2 ("5.0, 15.0, 30.0"); /* fF */
13  }
14
15  cell (INVX1) {
16    area : 1.764;
17    pin (A) { direction : input; capacitance : 1.42; }
18    pin (Y) {
19      direction : output;
20      function : "!A";
21      max_capacitance : 60.0;
22      timing () {
23        related_pin : "A";
24        timing_sense : negative_unate;
25        cell_rise (delay_template_3x3) {
26          values ( "0.045, 0.062, 0.091", \
27                "0.071, 0.094, 0.135", \

```

```

28         "0.108, 0.142, 0.198" );
29     }
30     cell_fall (delay_template_3x3) {
31         values ( "0.038, 0.054, 0.080", \
32                 "0.061, 0.083, 0.121", \
33                 "0.094, 0.127, 0.178" );
34     }
35 }
36 }
37 }
38 }

```

Two things deserve attention in that listing. First, the **units block at the top** sets the scale for every number below: delays are in **ns**, capacitances in **fF**. Misreading the unit is the single most common .lib bug junior engineers commit. Second, the **values** array is row-major, with rows indexed by **index_1** (input slew) and columns by **index_2** (output load).

3.3 Worked Example: Bilinear Interpolation by Hand

Let us take the **cell_rise** table from the listing above and ask: what is the cell delay if the input slew is **50 ps** and the output load is **15 fF**? The exact load (15 fF) sits on the grid, but the slew (50 ps) falls between 20 ps and 100 ps, so we have to interpolate along one axis.

Step 1: locate the bracketing grid points. Slew 50 ps lies between **index_1[0]=20** and **index_1[1]=100**. Load 15 fF is exactly **index_2[1]=15**.

Step 2: pull the four corner delays. Because load sits on the grid, only two corners matter:

$$D(20 \text{ ps}, 15 \text{ fF}) = 0.062 \text{ ns}, \quad D(100 \text{ ps}, 15 \text{ fF}) = 0.094 \text{ ns}.$$

Step 3: compute the slew fraction.

$$\alpha = \frac{50 - 20}{100 - 20} = \frac{30}{80} = 0.375.$$

Step 4: linearly blend.

$$D_{\text{interp}} = (1 - 0.375) \cdot 0.062 + 0.375 \cdot 0.094 = 0.625 \cdot 0.062 + 0.375 \cdot 0.094.$$

$$D_{\text{interp}} = 0.03875 + 0.03525 = 0.0740 \text{ ns} = \mathbf{74.0 \text{ ps}}.$$

So the tool reports a rise delay of **74 ps** for this INVX1 under those conditions. Now let us do the harder case where *both* axes are off-grid: slew 50 ps, load 22 fF.

Move 1. Pull all four corners:

$$\begin{aligned} D_{00} &= D(20, 15) = 0.062, & D_{01} &= D(20, 30) = 0.091, \\ D_{10} &= D(100, 15) = 0.094, & D_{11} &= D(100, 30) = 0.135. \end{aligned}$$

Move 2. Compute fractions on each axis:

$$\alpha = \frac{50-20}{100-20} = 0.375, \quad \beta = \frac{22-15}{30-15} = \frac{7}{15} = 0.4667.$$

Move 3. Interpolate along load at each slew value:

$$D(20, 22) = (1 - 0.4667) \cdot 0.062 + 0.4667 \cdot 0.091 = 0.0331 + 0.0425 = 0.0756.$$

$$D(100, 22) = (1 - 0.4667) \cdot 0.094 + 0.4667 \cdot 0.135 = 0.0501 + 0.0630 = 0.1131.$$

Move 4. Interpolate the two intermediate values along slew:

$$D = (1 - 0.375) \cdot 0.0756 + 0.375 \cdot 0.1131 = 0.0473 + 0.0424 = \mathbf{0.0897\text{ ns}} \approx \mathbf{90\text{ ps}}.$$

Move 5. Generalize to the abstract **bilinear interpolation formula**:

$$D(s, c) = (1 - \alpha)(1 - \beta) D_{00} + \alpha(1 - \beta) D_{10} + (1 - \alpha)\beta D_{01} + \alpha\beta D_{11}$$

where α is the fractional position along input slew and β is the fractional position along output load. The hand arithmetic above is exactly this formula, executed in two passes.

3.4 Setup, Hold, and the Other Tables

NLDM does more than store cell delays. Every sequential cell (flip-flops, latches) carries additional 2D tables that encode timing checks: a **setup** table indexed by data-slew and clock-slew, and a **hold** table with the same axes. The tool reads these to determine whether captured data is stable around the active clock edge. There are also **recovery** and **removal** tables for asynchronous pins, and **output transition** tables that propagate the slew downstream so the next gate's delay table can be looked up correctly.

Watch Out

Watch Out.

- **Units are not universal.** One foundry's .lib uses **1ns** and **pF**; another uses **1ps** and **fF**. Mixing tools that disagree on the units block produces silent errors of 1000x.
- **NLDM is not CCS.** Reading a .lib that claims CCS support but only filling in NLDM tables gives you NLDM accuracy with CCS file size. Verify by grepping for `output_current_rise`.
- **Hold tables are not setup tables.** A common bug: copying setup characterization data into the hold slot. Setup and hold have opposite signs and different reference edges.
- **PVT corners are not interchangeable.** The slow-slow (**WC**) .lib is for setup analysis; the fast-fast (**BC**) .lib is for hold. Loading only the typical (TC) corner will sign off a chip that fails silicon.

3.5 Why CCS Exists (and Why NLDM Quietly Lies at 7nm)

NLDM stores **one delay number** per (input slew, output load) grid point. That model rests on three assumptions, and every single one of them gets shakier as nodes shrink.

Assumption 1: the output load is a fixed lumped capacitor. It is not. Gate-to-drain coupling capacitance (the **Miller capacitance**) at the receiver gets multiplied by $(1 + A_v)$ where A_v is the receiver's small-signal gain during the transition. In a fast switching transition the effective load the driver feels is meaningfully larger than the static fanout capacitance the .lib was characterized against.

Assumption 2: the receiver's input pin capacitance is constant. It is not. PMOS and NMOS gate capacitance is voltage-dependent: a gate in cutoff presents less capacitance than the same gate in saturation. As the input ramps, the load looks different at 30% Vdd than at 70% Vdd. NLDM averages this and ships one number per pin. CCS stores the nonlinearity.

Assumption 3: only the slew (20%-to-80% or similar) matters, not the waveform shape. At 130 nm a slew was a clean, near-linear ramp. At 7 nm the upstream driver's output

is non-monotonic enough that two transitions with the same 20-80 slew can produce **different downstream delays**. NLDM cannot see this. CCS can, because it propagates the actual current waveform.

Key Idea

CCS replaces NLDM's single delay number with a **time-varying output current waveform per (slew, load) grid point**, so the next stage convolves *actual current shape* into its own response instead of inheriting a slew-only summary.

Worked example: when NLDM and CCS diverge. A buffer drives a fanout-of-4 load. NLDM says: static load = 4 input gates $\times$ 1.0 fF = 4.0 fF, look up (slew = 50 ps, load = 4 fF) in the table, get **delay = 32 ps**. CCS sees that the receiver's effective Miller-multiplied capacitance during the transition is closer to $4.0 \times 1.4 = 5.6$ fF, AND that the receiver pin cap rises by 20% as the input crosses the switching threshold. The CCS-computed delay is **closer to 42 ps**. **A 30% error on a single stage, compounded over a 15-stage path, is the difference between meeting timing and missing it by 100 ps.**

ECSM (Cadence) encodes the same physics with a voltage-vs-time waveform instead of current-vs-time. Same intent, different storage. Innovus consumes ECSM natively; PrimeTime consumes CCS. Foundries ship both.

File size cost. A CCS .lib is roughly 5 to 10x the size of the equivalent NLDM library because every (slew, load) entry now stores a sampled current waveform (typically 20 time points) instead of one delay scalar. This is why the .db (binary compiled .lib) matters: loading 200 CCS views as ASCII text would burn an hour on Liberty parsing alone.

Sidebar: optional depth, skip if tired

Why MMMC makes this worse. CCS file size multiplied by MMMC view count is what actually fills your scratch disk. A 5 nm block with 200 .lib views, each in CCS form at 80 MB, lands at 16 GB of library data per run. The binary .db form compresses this maybe 3x. **This is why the LIBPATH variable in Innovus matters and why corrupted .db caches eat days.**

Why PD Cares

- **Arm internship lens.** When a designer hands you a failing path, the first reflex is to open the .lib for the offending cell and confirm its drive strength, max-cap, and the actual delay numbers at the path's slew and load. Half the bugs assigned to junior PD engineers turn out to be a misloaded corner library.
- **Foundry-flow PD engineer lens.** You will routinely load **8 to 32 libraries** per run under MMMC: WC, TC, BC across two voltage rails and three temperatures, times functional mode (sleep, active, scan). Knowing which corner pessimizes setup vs hold determines which library a slack violation belongs to and where the fix has to land.
- **VLSI interview lens.** Expect "*walk me through what's inside a .lib file*" and "*why CCS over NLDM at advanced nodes?*" as warm-up questions. The strong answer names the units block, the lookup template, the timing arc with related `_pin` and `_sense`, and finishes with Miller capacitance and receiver-pin modeling as the CCS justification.

Checkpoint

You can now (1) name the four things a .lib stores per cell (pins, function, timing arcs, power), (2) read an NLDM lookup table and bilinearly interpolate a delay by hand, (3) explain why CCS exists and what it adds, and (4) defend why a real flow loads many .lib files under MMMC rather than just one.

Say it in 2 seconds: *Slew in, load out, table in the middle, corner on the side.*

Chapter 4

The LEF File: Physical Abstraction of Cells and Layers

TL;DR

- **Last chapter:** the `.lib` stored cell behavior as NLDM lookup tables across PVT. **This chapter:** the LEF stores the same cell's physical footprint, so the placer can put it somewhere legal without ever opening the GDS.
- The **LEF (Library Exchange Format)** file gives the placer and router the **physical view** of every cell that the timing `.lib` characterized: bounding box, pin shapes, blockages, and which layer each pin lives on.
- LEF comes in two flavors that get shipped separately: a **technology LEF** (one per process node, describing metal layers, vias, and design rules) and a **cell LEF** (one per standard cell library, describing each MACRO).
- LEF deliberately throws away the transistor-level GDS. The placer never sees diffusion or poly. It sees rectangles on metal layers and a placement **SITE** grid. GDS only re-enters the flow at LVS and tape-out.
- Inside a MACRO: **SIZE** (width $\times$ height), **SYMMETRY**, **SITE**, one **PIN** block per logical pin (with **PORT** geometries on specific **LAYERS**), and **OBS** blockages that the router must avoid.

4.1 Why the placer is blind to your beautiful layout

Your standard cell layout is a stick diagram with diffusion, poly, contacts, and three or four metal layers. It is gorgeous. The placer does not care. When the placer drops an `INV_X1` into a row, it is moving a small **rectangle of width W and height H** , snapping its origin to a **SITE** on a row, and asking one question: *do the pin shapes on M1 line up with a routing track?*

Everything else (the FET widths, the poly pitch, the well taps) is **stripped out** before the cell ever reaches P&R. The file that does the stripping is the LEF.

Memory hook

LEF is the placer's passport photo. The cell's full GDS face stays home; only the **rectangle, the pins, and the rails** travel through P&R. If the passport photo is wrong, the cell still gets stopped at the border (DRC).

Memory hook

Tech-LEF paints the streets; cell-LEF parks the cars. The tech-LEF declares which metal layers exist, their pitch, and their direction. The cell-LEF tells the tool where

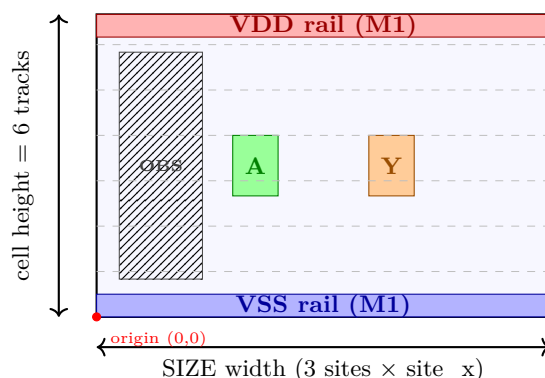


Figure 4.1: Abstract physical view of an `INV_X1` as seen through the LEF. The placer sees this rectangle, the VDD/VSS rails, the M1 pin shapes for A and Y, and the OBS blockage. It never sees the transistors underneath.

each cell parks on those streets. Mismatched grids mean the cars do not fit between the curbs.

Gut check

A placer is dropping an `INV_X1` into a row. Which of these does it read from the LEF, and which from the `.lib`? (a) cell delay vs. load, (b) cell width in microns, (c) pin A input capacitance, (d) which metal layer pin A sits on.

LEF: (b) and (d). Timing .lib: (a) and (c). LEF is physical only. The .lib is electrical only. They share cell names so the tool can join the two views.

Gut check

You open a cell LEF and see `SIZE 0.6 BY 1.2`. Where does the unit (microns vs. database units) come from?

The technology LEF, via the `UNITS` section (typically `DATABASE MICRONS 1000` or similar). The cell LEF inherits the unit; it does not redeclare it.

Pause

So far: LEF abstracts the physical view, ships in two pieces (tech + cell), and intentionally hides the GDS. Next we nail down the one sentence that justifies everything else in this chapter.

Key Idea

The LEF is the **contract between the cell library and the placer/router**: it promises that a cell will occupy exactly this footprint, expose pins on exactly these layers at exactly these coordinates, and block routing in exactly these regions, so the tool can resolve placement and routing **without ever opening the GDS**.

4.2 Anatomy of a MACRO block

A cell LEF is a stack of `MACRO` blocks. Each block declares the cell name, its bounding box, what kind of cell it is (`CLASS CORE`, `PAD`, `BLOCK`, etc.), which placement site it snaps to, and then one `PIN` sub-block per logical pin plus an `OBS` sub-block for blockages.

A few items deserve a closer look. **SYMMETRY X Y** tells the placer it may mirror this cell about either axis (commonly used to flip every other row so VDD/VSS rails abut). **SITE CoreSite** ties the cell to a placement site defined in the tech-LEF; the cell's width must be an integer multiple of the site width. **OBS** is a router-only directive that says *do not route here*, typically used to hide internal M1 jumpers from the routing tool.

4.3 Worked example: site arithmetic for an INV__X1

Move 1. Given: cell **SIZE 0.6 BY 1.2** (microns). Tech-LEF declares **SITE CoreSite SIZE 0.2 BY 1.2**. We want: how many sites the cell occupies horizontally, its area, and whether it aligns to the row grid.

Move 2. Sites occupied horizontally: $N_{\text{sites}} = \frac{W_{\text{cell}}}{W_{\text{site}}} = \frac{0.6}{0.2} = 3$ sites. The cell consumes a contiguous run of **three placement sites**.

Move 3. Row alignment: row height is the site height = 1.2 μm , and the cell height is also 1.2 μm . Match. The cell fits exactly in one row, with its VDD pin on the top edge and VSS on the bottom.

Move 4. Cell area: $A = W \times H = 0.6 \times 1.2 = 0.72 \mu\text{m}^2$. This is what the placer reports for utilization accounting, not the active silicon area.

Move 5. Sanity check: if a designer had written **SIZE 0.55 BY 1.2**, then $0.55/0.2 = 2.75$ sites. The placer would either reject the macro at library load (*cell width not a multiple of site width*) or silently round up to 3 sites, wasting 0.05 μm of width per instance. Multiply that by half a million instances and you have lost a chunk of die area to a bad library.

Move 6. Conclusion: a healthy library always reports **SIZE** widths that are exact integer multiples of the **SITE** width, and the cell's row height equals the row pitch. **Both equalities are silently assumed by the placer.**

Watch Out

Four ways a LEF will silently wreck your block:

- **Tech-LEF vs. cell-LEF confusion.** Loading two tech-LEFs (e.g., one from the foundry and one from the IP vendor) causes contradictory **LAYER** definitions. The tool may pick the wrong one without warning.
- **M1 pin access at advanced nodes.** At 7nm and below, M1 is so dense that a pin **RECT** may have zero legal access points after considering via rules. The router fails to connect it, even though DRC on the cell alone passes.
- **OBS misuse.** An overly aggressive **OBS** on M2 forces all routes to detour, blowing up congestion. Too little **OBS** and the router crosses an internal node, creating shorts at LVS.
- **Mismatched SITE definitions.** The std cell library was characterized on **SITE CoreSite** (0.2×1.2), but the floorplan rows were generated on a slightly different site. Cells will not snap, and the tool emits thousands of placement-legality errors.

Sidebar: optional depth, skip if tired

Tech-LEF vs. cell-LEF, and where LEF came from. The **tech-LEF** is shipped by the foundry (or PDK provider) once per process node. It declares units, the metal stack

(layer names, directions, pitches, min width, spacing), via definitions, and the placement **SITE** grid. The **cell-LEF** is shipped per standard cell library and contains one **MACRO** per cell. Both files are plain ASCII, historically defined by **Cadence** in the late 1980s as an open interchange format paired with **DEF** (Design Exchange Format) for post-placement data. The pair (LEF/DEF) is still the lingua franca that lets a Synopsys placer eat a TSMC library or a Cadence router eat a Samsung library without binary glue.

Why PD Cares

- **Arm PD intern.** When a new tech drop lands, the first thing you do is diff the tech-LEF against the previous version: layer pitches, via rules, **SITE** sizes. A silent pitch change forces a full re-floorplan.
- **Generic PD engineer.** If a block has unroutable nets concentrated around one cell type, open that cell's LEF and inspect its M1 pin **RECTs** and **OBS** regions. Pin-access problems are visible at the library level long before they explode in the router log.
- **VLSI interview.** Expect the question: *What is in a LEF file, and how is it different from a GDS?* The crisp answer is: LEF is the **abstract physical view** (bounding box, pins, blockages, on metal layers only) used by P&R; GDS is the **full polygon-level layout** used only at LVS and tape-out.

Checkpoint

You should now be able to: (1) state what LEF abstracts away and why, (2) name the two LEF flavors and what each contains, (3) read a **MACRO** block and identify **SIZE**, **SITE**, **PIN**, **PORT**, **LAYER**, **OBS**, (4) compute site occupancy for a given cell width.

Say it in 2 seconds: *“**SIZE** the cell, **SITE** the row, **PIN** the layer, **OBS** the rest.”*
Four verbs map one-to-one onto the four LEF keywords the placer/router actually consults on every cell.

```

1 MACRO INV_X1
2   CLASS CORE ;
3   ORIGIN 0 0 ;
4   SIZE 0.6 BY 1.2 ;
5   SYMMETRY X Y ;
6   SITE CoreSite ;
7
8   PIN A
9     DIRECTION INPUT ;
10    USE SIGNAL ;
11    ANTENNAGATEAREA 0.14 ;
12    PORT
13      LAYER M1 ;
14      RECT 0.18 0.40 0.24 0.80 ;
15    END
16  END A
17
18  PIN Y
19    DIRECTION OUTPUT ;
20    USE SIGNAL ;
21    PORT
22      LAYER M1 ;
23      RECT 0.36 0.40 0.42 0.80 ;
24    END
25  END Y
26
27  PIN VDD
28    DIRECTION INOUT ; USE POWER ;
29    PORT LAYER M1 ; RECT 0.00 1.13 0.60 1.20 ; END
30  END VDD
31
32  PIN VSS
33    DIRECTION INOUT ; USE GROUND ;
34    PORT LAYER M1 ; RECT 0.00 0.00 0.60 0.07 ; END
35  END VSS
36
37  OBS
38    LAYER M1 ; RECT 0.05 0.10 0.14 1.10 ;
39  END
40 END INV_X1

```

Figure 4.2: A cell-LEF `MACRO` excerpt for an `INV_X1`. Coordinates are in microns (set by the tech LEF). Pins A and Y expose M1 rectangles; VDD/VSS stripe the top and bottom; OBS blocks a region of M1 from the router.

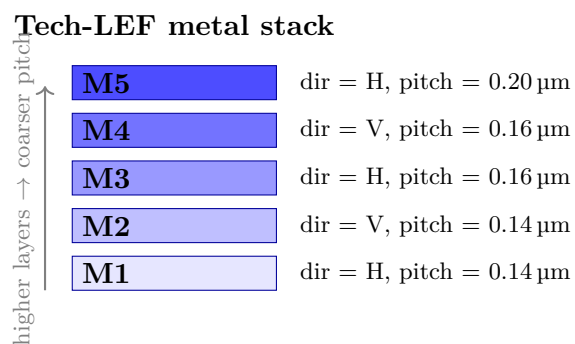


Figure 4.3: A simplified tech-LEF view of the metal stack. Each `LAYER` declaration carries a `DIRECTION` (horizontal or vertical preferred routing) and a `PITCH`. The router uses these to lay tracks; the cell-LEF places its pin `RECTs` so they land on those tracks.

Chapter 5

The Constraints File (SDC): Telling the Tool What “Working” Means

TL;DR

- **Last chapter:** LEF was the contract between the library and the placer, all physical and no time. **This chapter:** SDC is the contract between the designer and every timing-aware tool downstream, all time and no physics.
- **SDC** (Synopsys Design Constraints) is a **Tcl-based contract** that tells synthesis, STA, and PD what timing, power, and area the chip must hit.
- The **Big 5** commands are `create_clock`, `set_input_delay`, `set_output_delay`, `set_clock_uncertainty`, and `set_false_path` / `set_multicycle_path`.
- **Setup** asks “did data arrive *before* the next capture edge minus the setup window?” **Hold** asks “did data wait *long enough past* the current edge so the previous value was safely captured?”
- Slack is the headline number. Positive means met. Negative means the tool has work to do, or you have an SDC bug.

Opening: the contract nobody reads until it breaks

A netlist by itself is silent about **when** signals should arrive. You can hand the same gate-level netlist to two engineers, give them two different SDC files, and they will ship two completely different chips. One will close at 1 GHz; the other will look broken even though every transistor is identical. The constraints file is the difference.

This chapter pulls apart the SDC **piece by piece**. We will define clocks, anchor the boundary with input and output delays, do the slack arithmetic by hand for both setup and hold, and look at where engineers most often shoot themselves in the foot. By the end the file should stop feeling like a wall of Tcl and start feeling like a small, opinionated specification.

5.1 What lives inside the SDC

The SDC bundles three flavors of constraint into one file. **Timing** constraints define the clocks and the I/O delays that anchor every path. **Optimization** constraints (max transition, max capacitance, max fanout, input/output load) shape how the tool buffers and sizes the design. **Power** constraints cap dynamic and leakage budgets, often paired with a SAIF switching-activity file.

The lecturer is blunt about why SDC matters: it is read by **synthesis** (Design Compiler), by **STA** (PrimeTime), by **place-and-route** (ICC2 / Innovus), and by **formal** flows. Every downstream tool trusts it. If the SDC lies, every report based on it also lies.

5.1.1 The timing waveform: launch, capture, slack

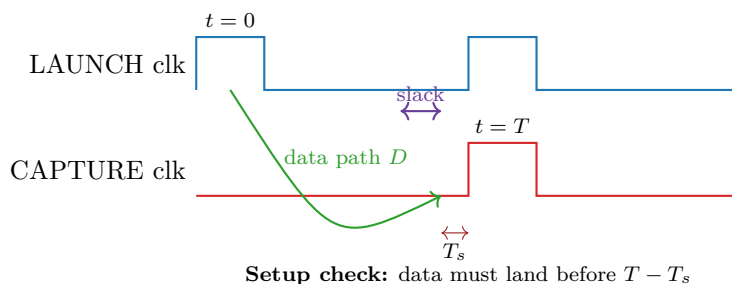


Figure 5.1: Setup check on a single launch/capture pair. $\text{Slack} = T + \text{skew} - D - T_s$.

The launch flop fires its Q output at $t = 0$. The signal walks through some combinational cloud, taking total delay D . The capture flop **samples** at $t = T$, but it needs the data to be stable for a small window T_s **before** that edge. Anything left over is positive slack.

Memory hook

“Launch lights the fuse; capture clips the wire.” If the fuse burns too slowly (large D), capture’s scissors slice through a signal still in motion. Setup violations are missed deadlines.

5.1.2 Anchoring the boundary: input and output delay

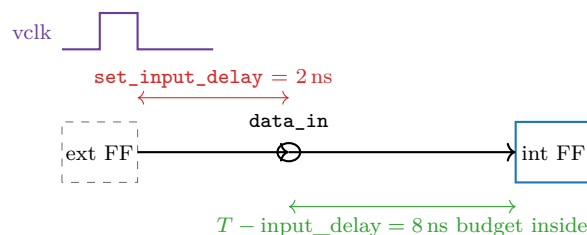


Figure 5.2: `set_input_delay` models the time the signal already spent outside the block before reaching the port. The internal budget is what is left of the clock period.

Input delay says: “by the time the signal walks in through this port, X ns of the clock period have already been consumed upstream.” **Output delay** says: “after the signal walks out through that port, the downstream flop still needs Y ns to capture it.” Both are referenced to a clock, real or virtual, so STA knows what edge to measure against.

Memory hook

“Input delay is rent paid before you moved in.” The clock period is your apartment; the upstream logic already used some of the square footage. Whatever is left is yours to optimize.

5.2 The Big 5 SDC commands, by example

5.2.1 create_clock and create_generated_clock

Listing 5.1: Defining a primary clock and a divide-by-2 generated clock.

```

1 # Primary 1 GHz clock on the CLK port
2 create_clock -name CLK_SYS -period 1.0 -waveform {0 0.5} \
3   [get_ports CLK]
4
5 # Divide-by-2 generated clock on an internal flop's Q
6 create_generated_clock -name CLK_DIV2 -source [get_ports CLK] \
7   -divide_by 2 [get_pins clkgen/div2_reg/Q]

```

A **primary clock** declares period and waveform at a port or pin. A **generated clock** is derived from a master with a known divide, multiply, or edge relationship. If you forget to declare a generated clock, STA will silently treat that internal node as just data and your reports will be wrong.

5.2.2 set_input_delay, set_output_delay, virtual clocks

Listing 5.2: Boundary I/O timing referenced to a virtual clock.

```

1 # Virtual clock used only for I/O timing (no real pin)
2 create_clock -name VCLK -period 1.0
3
4 # Data arrives 0.3 ns after VCLK edge
5 set_input_delay 0.3 -clock VCLK [get_ports data_in*]
6
7 # Downstream FF needs 0.4 ns before its clock edge
8 set_output_delay 0.4 -clock VCLK [get_ports data_out*]

```

Virtual clocks exist purely as a timing reference; no pin in the block actually toggles with them. They are the standard trick when the real launching/capturing flop lives in another block and you only want to constrain the interface.

5.2.3 set_clock_uncertainty, set_false_path, set_multicycle_path

Listing 5.3: Margin, exceptions, and multicycle relief.

```

1 # Account for jitter + a safety margin (pre-CTS uses larger value)
2 set_clock_uncertainty -setup 0.08 [get_clocks CLK_SYS]
3 set_clock_uncertainty -hold 0.03 [get_clocks CLK_SYS]
4
5 # Async config path: ignore for timing entirely
6 set_false_path -from [get_pins cfg_reg*/Q] -to [get_pins core/*/D]
7
8 # Two-cycle path: data is allowed 2T to settle, hold relative to launch
9 set_multicycle_path 2 -setup -from [get_pins slow_src/Q] -to [get_pins slow_dst/D]
10 set_multicycle_path 1 -hold -from [get_pins slow_src/Q] -to [get_pins slow_dst/D]

```

Uncertainty folds jitter, skew estimate, and pessimism into one number that is subtracted from the available period. Pre-CTS the value is large because the clock tree does not exist yet; post-CTS it shrinks. **False paths** declare “never time this.” **Multicycle paths** say “this path is allowed N clock periods instead of one.”

Gut check

The SDC has `create_clock -period 2.0` on CLK and `set_clock_uncertainty -setup 0.15`. What is the effective period available for data-path delay before subtracting setup time and input/output delays?

$T_{\text{eff}} = 2.0 - 0.15 = 1.85 \text{ ns}$. *Uncertainty is consumed before any logic delay.*

Gut check

Why must a generated clock be *declared* even though it is physically derived from a real flop driven by a real clock?

Because STA needs to know the period and edge relationship so it can correctly time paths that launch on the master and capture on the generated clock (or vice versa). Without the declaration, STA treats the node as data, not as a clock, and reports nonsense.

Pause

So far: SDC = Tcl contract. The Big 5 are clock, input delay, output delay, uncertainty, and exceptions. Setup is “arrive before deadline,” hold is “do not change too early.” Next we do the math.

Key Idea

The SDC is the **single source of truth** that converts a netlist into a timed promise: every downstream tool (synthesis, STA, PnR, signoff) measures success against the same clock periods, I/O delays, uncertainties, and exceptions you write here.

5.3 Worked example: setup slack arithmetic

A single capture-to-launch pair with: launch edge at $t = 0$, period $T = 1.0 \text{ ns}$, combinational delay $D = 0.8 \text{ ns}$, capture clock skew $\text{skew} = +0.05 \text{ ns}$ (capture clock arrives *later* than launch clock, helping setup), and capture flop setup window $T_s = 0.10 \text{ ns}$.

Move 1. Write the setup-slack formula with numbers *first*.

$$\text{slack}_{\text{setup}} = T + \text{skew} - D - T_s = 1.0 + 0.05 - 0.8 - 0.10$$

Move 2. Add the helpers, subtract the burdens.

$$\text{slack}_{\text{setup}} = 1.05 - 0.90 = 0.15 \text{ ns}$$

Move 3. Positive. Setup is **met** with 150 ps of margin.

Move 4. Generalize. The setup check at any endpoint is

$$\text{slack}_{\text{setup}} = (T_{\text{capture}} - T_{\text{launch}}) + \text{skew} - D_{\text{data}} - T_s - U$$

where U is clock uncertainty.

Now hold. Same path, but hold time $T_h = 0.05 \text{ ns}$, and assume minimum data delay $D_{\text{min}} = 0.20 \text{ ns}$ from the same launch edge.

Move 5. Hold check arithmetic:

$$\text{slack}_{\text{hold}} = D_{\text{min}} - \text{skew} - T_h = 0.20 - 0.05 - 0.05 = 0.10 \text{ ns}$$

Positive. Hold is also met. Note that the same skew that *helped* setup *hurts* hold. This is why fixing one violation often breaks the other.

Move 6. Generalize:

$$\text{slack}_{\text{hold}} = D_{\text{min}} - \text{skew} - T_h - U_{\text{hold}}$$

Move 7. Sanity. If skew were +0.25 ns instead, hold slack would be $0.20 - 0.25 - 0.05 = -0.10$ ns: a hold violation, even though setup got easier. Skew has opposite signs in the two equations. Memorize that.

Watch Out

The four most common SDC mistakes that get interns roasted in review:

- **Missing generated clocks.** The divider's Q is treated as data, and downstream paths look magically clean (they are not).
- **No input/output delay on boundary ports.** STA assumes 0 ns, so the inside-the-block budget looks like a full period. Real silicon then fails.
- **Wrong reference clock on set_input_delay.** You constrain against CLK_A but the upstream block actually launches on CLK_B. Slack numbers are fiction.
- **Using set_false_path as a hammer.** Engineers slap false paths on violating endpoints to make the report green. The path is still real silicon. The chip still fails. Use false paths only for genuinely asynchronous or unrelated logic.

Sidebar: MMMC, modes, and corners

Real chips do not live in one SDC. **MMMC** (Multi-Mode Multi-Corner) means the design is closed across a matrix of **modes** (functional, scan-shift, scan-capture, test, low-power) and **corners** (slow-slow / fast-fast process, high/low voltage, hot/cold). Each mode often has its own SDC because scan-shift typically uses a much slower clock and a different topology than functional mode. Signoff means every (mode, corner) pair closes timing simultaneously. This is where SDC discipline pays for itself, because a forgotten **set_case_analysis** on the scan-enable can silently disable timing across half the design.

Why PD Cares

- **Arm PD intern:** you will read SDC files on day one and write SDC tweaks by week two. Reviewers will ask you to justify every false path you add. Bring evidence, not vibes.
- **Generic foundry PD engineer:** MMMC closure is your full-time job. You spend more hours in PrimeTime reading SDC-driven reports than you do in the layout viewer. Tool-agnostic SDC fluency travels across Synopsys, Cadence, and Siemens flows.
- **VLSI interview:** this is **the** most asked topic in PD and STA interviews. Expect “write the setup and hold slack equations,” “what is a virtual clock,” “when do you use multicycle vs false path,” and “draw the launch/capture waveform.” If you can produce the equations and the diagram from this chapter on a whiteboard cold, you

are ahead of 80% of new grads.

Checkpoint

You can now (a) name the Big 5 SDC commands and what each one anchors, (b) write the setup and hold slack equations and substitute numbers without looking them up, (c) explain why skew helps setup but hurts hold, and (d) spot the four common SDC mistakes in a code review. **Say it in 2 seconds: *Clock, in, out, jitter, exception. Setup arrives early; hold stays late.***

Chapter 6

The Technology File: Where Physics Becomes Rules

TL;DR

- **Last chapter:** the SDC turned a netlist into a timed promise every tool measures against. **This chapter:** the tech file is the foundry rulebook every routed shape must obey before that promise can be cashed in silicon.
- The **technology file** is the foundry contract that turns silicon physics into machine-readable design rules your PD tool obeys layer by layer.
- It carries **layer definitions, min width/spacing, min area, via geometries, sheet resistance, thickness, permittivity, and colors** for multi-patterning.
- Without it, the router has **no legal moves**: every wire it draws would be a DRC violation waiting to happen.
- Two cousins exist in real flows: the older Cadence **.tf** format and the modern **tech-LEF** abstraction, often paired with an ITF for parasitic extraction.

Memory hook

Imagine handing a contractor a blueprint with no building code, no minimum stud spacing, no fire rating, no permitted materials. He builds whatever he wants and the inspector burns it down. That is exactly what a place-and-route tool would do without a tech file, except the inspector is the foundry mask shop and “burning it down” means a \$3M tapeout that yields zero working die. So how does a four-kilobyte ASCII file save a multi-million-dollar wafer run?

6.1 The Stack You Are Actually Routing On

A modern CMOS chip is a sandwich of conducting metal layers separated by dielectric, stitched together by tiny vertical plugs called vias. The tech file describes **every one of those layers**, bottom to top, with the numbers a router needs to keep wires legal and signals fast. Layers near the substrate are **thin and tight** (good for short local nets, bad for current); layers up top are **thick and wide** (good for power and clock, bad for density).

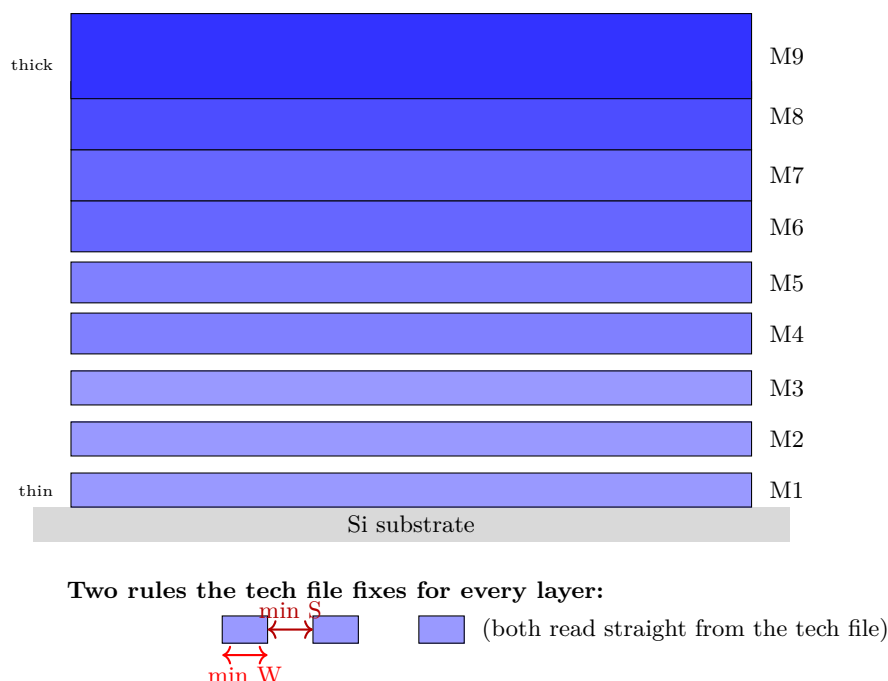


Figure 6.1: Metal stack M1 through M9. Lower layers are thin and dense; upper layers grow thicker for current handling. The red arrows on M2 show the two rules the tech file enforces everywhere: minimum width and minimum spacing.

Key Idea

The technology file is the **foundry-supplied rulebook** that names every physical layer, fixes its geometric and electrical parameters, and tells the PD tool exactly what shapes it is allowed to draw.

6.2 What Lives Inside the File

Open a real tech file and you will see a layered structure: chip-level parameters first (chip X/Y, frequency, FFT area for RF flows), then a substrate and dielectric block, then a long tail of **metal and via blocks**. Each metal entry carries a layer number, a name, a color, sheet resistance, thickness, and the design rules that come with it. The dielectric entries carry **permittivity** (relative, unitless) and **thickness** in microns, because that is what an extractor needs to back out capacitance.

Listing 6.1: Skeleton of a tech file fragment (illustrative)

```

1 chip_x = 1200
2 chip_y = 1200
3 tech_file = "rf28_v1.tf"
4 frequency = 2.4e9
5 layer substrate { rho = 10 ; thickness = 280 }
6 layer ild1 { eps_r = 4.2 ; thickness = 0.4 }
7 layer metal0 { num=1 ; rs=0.12 ; t=0.13 ; color=red ; minW=0.07 ; minS=0.07 }
8 layer metal1 { num=2 ; rs=0.10 ; t=0.15 ; color=green ; minW=0.07 ; minS=0.07 }
9 via v1 { from=metal0 ; to=metal1 ; cut=0.07x0.07 ; enc=0.005 }
```

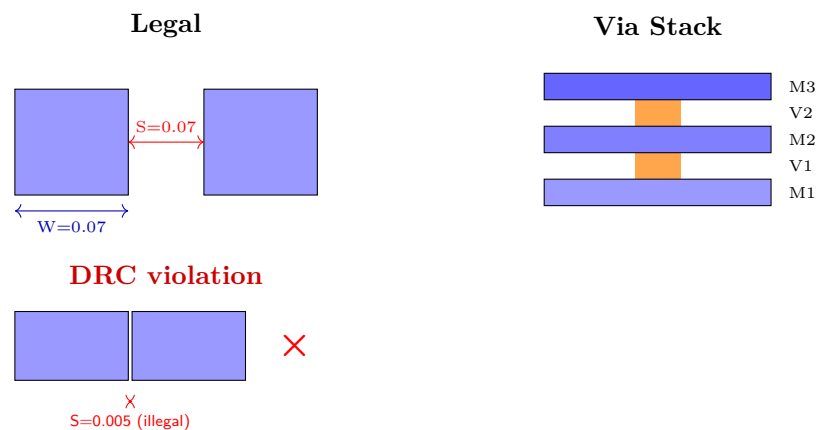


Figure 6.2: Left: two M2 wires with the legal minimum spacing the tech file declares. Right: a via stack $M1 \rightarrow V1 \rightarrow M2 \rightarrow V2 \rightarrow M3$ with the vias defined as small square cuts inside larger metal landing pads. Bottom: the same two wires drawn too close, which the DRC engine flags instantly because the spacing rule was read straight from the tech file.

Memory hooks

Why PD Cares

Hook 1. The **foundry stapler** *slams* the rulebook onto your desk; lose it and your router *flies blind into a million-dollar wall*.

Hook 2. **Dielectric layers** *whisper* their permittivity to the **extractor**; misread them and your *timing report lies to your face*.

Gut checks

Gut check

You delete the via definition for V3 from the tech file and rerun routing. What breaks first?

The router cannot legally connect M3 to M4, so any net needing that transition fails (open net or DRC) and the tool errors out before completing.

Gut check

Why is M9 drawn three times thicker than M1 in the stack figure?

Upper metals carry power and global clock; thicker metal lowers sheet resistance and IR drop. Lower metals stay thin to keep pitch tight for dense local routing.

Pause

Stop. Before reading on, predict: if a tech file lists **only** M1 through M5 but your floorplan needs a top-level clock mesh, what happens when you try to route the mesh on M6? Say it out loud.

6.3 Worked Example: Counting Tracks Across a Channel

Setup. M2 has **min width** $W = 0.07 \mu\text{m}$ and **min spacing** $S = 0.07 \mu\text{m}$. A routing channel is **5.0 microns wide**. How many M2 tracks fit?

Move 1. Substitute the numbers into the pitch definition:

$$\text{pitch} = W + S = 0.07 + 0.07 = 0.14 \mu m.$$

Move 2. Divide the available channel width by the pitch:

$$N = \left\lfloor \frac{5.0}{0.14} \right\rfloor = \lfloor 35.71 \rfloor = 35 \text{ tracks.}$$

Move 3. Sanity check by rebuilding the total occupied width:

$$35 \times 0.14 = 4.90 \mu m \leq 5.0 \mu m. \checkmark$$

Move 4. The leftover $0.10 \mu m$ is not enough for a 36th track (would need another $0.14 \mu m$), so 35 is tight and correct.

Key Idea

Routing capacity per layer is set entirely by the tech file. Change **minW** or **minS** by ten nanometers and your channel either fits the design or fails routing congestion.

6.4 Watch Out

Watch Out

Tech file vs tech-LEF. They are siblings, not twins. The Cadence **.tf** is the older, richer foundry-format file (often paired with an **ITF** for extraction). **tech-LEF** is the LEF/DEF-style abstraction modern Synopsys and tool-agnostic flows consume. Some tools accept either depending on the flow; do not assume one is a drop-in replacement for the other. Always match the tech format to what your PnR tool expects in its reference flow doc.

Watch Out

Colors are not just metal names. At advanced nodes (sub-20nm), the **color** field is not cosmetic. It encodes **mask assignment for multi-patterning**. Two same-color shapes that are too close are a **coloring violation**, distinct from a normal spacing DRC. Treat color as a hard physical attribute, not a label.

Sidebar: optional depth, skip if tired

Why advanced nodes make this file explode At 7nm and below, a single metal layer is printed across **multiple masks** (LELE, SADP, SAQP). The tech file now has to declare **decomposition rules**, **coloring constraints**, **cut-mask geometries**, and per-color minimum spacings. A 28nm tech file might be a few thousand lines; a 5nm tech file plus its rule decks pushes into the hundreds of thousands. The discipline is the same, the size is not.

Why PD Cares

Why PD cares (three lenses):

- **Arm PD intern:** You will never write a tech file, but on day one you will load one and **grep** it for layer counts, min pitches, and via definitions to sanity check your floorplan.

- **Generic foundry PD engineer:** You consume the PDK as ground truth. When a router complains “no legal via found,” the answer is in the tech file’s via section nine times out of ten.
- **VLSI interview:** “What is in a tech file and why do you need it?” is a layup question. Name layer defs, design rules, vias, RC parameters, and units. Bonus points for distinguishing `.tf` from tech-LEF.

Checkpoint

You should now be able to:

- Name at least five categories of information stored in a tech file.
- Compute routing pitch and tracks per channel from min width and min spacing.
- Explain the difference between `.tf` and tech-LEF in one sentence.
- Say why coloring fields stop being decorative at advanced nodes.

Say it in 2 seconds: *LIVR-C: Layers, ILD/permittivity, Vias, Rules (W/S/area), Colors. Five buckets, one file, every PD flow.*

Chapter 7

The DEF File: A Snapshot of Where Everything Lives

TL;DR

- **Last chapter:** the tech file listed every legal shape on every layer. **This chapter:** the DEF records exactly which legal shapes ended up where, snapshot by snapshot through the flow.
- **DEF** (Design Exchange Format) is an ASCII file that captures the **current physical state** of the design: die shape, rows, instance placement, IO pin locations, and (later) full routing.
- DEF is emitted at **every major PD checkpoint** (post-floorplan, post-place, post-CTS, post-route); same file format, increasing completeness at each stage.
- **LEF + DEF together** give you enough to fully visualize a design in a layout viewer with no GDS in sight; GDS is for the fab, DEF is the working format used *during* PD.

Memory hook

You have a netlist that says *what* the design is and a LEF that says *what each cell looks like*. Hand both to a tool and it still has no idea where to put anything. Somewhere there has to be a file that says “**this exact instance sits at (12.5, 7.5) microns, oriented north, and is fixed.**” That file gets rewritten after floorplan, after place, after CTS, after route. Same name. Same format. Different snapshot. What is inside it?

7.1 What DEF Actually Is

DEF stands for **Design Exchange Format**. It is a plain-text file that describes the physical structure of a specific design instance: the die outline, the rows that standard cells slot into, the placement of every component, the IO pin locations on the boundary, and (after routing) the full wire and via geometry.

LEF tells the tool what a NAND2 cell physically looks like in the abstract. DEF tells the tool that instance `u_alu/g_carry` of that NAND2 sits at coordinate (12500, 7500) facing north. One is library, one is design.

Key Idea

A DEF file is a **textual snapshot of the design's physical state** at one point in the PD flow; LEF describes cells in the abstract, DEF describes **this design's instances** in concrete coordinates.

7.2 The Floorplan View

The clearest mental picture of what DEF encodes is the floorplan itself. A die boundary, a core area inside it, IO pins around the edge, a few hard macros pre-placed, and standard cell rows filling the rest. Every one of those geometric facts has a DEF section behind it.

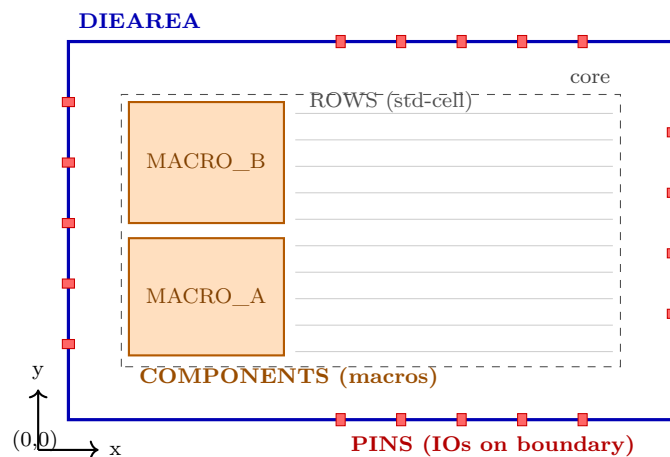


Figure 7.1: Floorplan-style picture of what a DEF file encodes. The die outline is the **DIEAREA** polygon, two pre-placed hard macros and the standard-cell rows live in **COMPONENTS** and **ROWS**, and the IO ports on the boundary are **PINS**. Each visible thing in this picture maps to one DEF section.

The DIEAREA polygon is walked **clockwise from the origin**, and order matters; reversing it flips the inside/outside semantics. The rows tile the inner core. The macros sit at fixed coordinates that the floorplanner has already committed to. The IO pads sit on the die edge.

Memory hook

Picture a **stage manager's set diagram** taped backstage: every prop (instance) has an X mark, every entrance (pin) is on the perimeter, every dancer's lane (row) is chalked across the floor. Lose the diagram mid-show and the next scene change collides two macros, which in silicon means DRC errors before you can spell "re-floorplan."

7.3 Inside the File

Here is what the relevant sections look like in the actual ASCII. Names are illustrative; the keywords (DIEAREA, ROW, COMPONENTS, PINS, NETS) are not.

Listing 7.1: Skeleton of a DEF file. Each major section ends with an explicit END keyword.

```

1 VERSION 5.8 ;
2 DIVIDERCHAR "/" ;
3 BUSBITCHARS "[]" ;
4 DESIGN top_block ;
5 UNITS DISTANCE MICRONS 1000 ;

```

```

6
7 DIEAREA ( 0 0 ) ( 0 38000 ) ( 14000 38000 )
8   ( 14000 22000 ) ( 18000 22000 ) ( 18000 0 ) ;
9
10 ROW ROW_0 CORE 100 100 N DO 420 BY 1 STEP 200 1400 ;
11
12 COMPONENTS 4 ;
13   - ENDCAP_L_0 TAPCELL_X1 + FIXED ( 100 100 ) N ;
14   - u_alu/g_carry NAND2_X1 + PLACED ( 12500 7500 ) N ;
15   - u_alu/g_sum XOR2_X2 + PLACED ( 13200 7500 ) FN ;
16   - u_ctrl/fsm_q DFFR_X1 + PLACED ( 4800 9100 ) S ;
17 END COMPONENTS
18
19 PINS 3 ;
20   - clk + NET clk + DIRECTION INPUT + USE SIGNAL
21     + LAYER M4 ( -70 0 ) ( 70 140 )
22     + FIXED ( 9000 38000 ) N ;
23   - rst_n + NET rst_n + DIRECTION INPUT + USE SIGNAL
24     + LAYER M4 ( -70 0 ) ( 70 140 )
25     + FIXED ( 11000 38000 ) N ;
26   - q_out + NET q_out + DIRECTION OUTPUT + USE SIGNAL
27     + LAYER M4 ( -70 0 ) ( 70 140 )
28     + FIXED ( 18000 12000 ) E ;
29 END PINS
30
31 NETS 2 ;
32   - n_carry ( u_alu/g_carry Z ) ( u_alu/g_sum A ) ;
33   - clk ( PIN clk ) ( u_ctrl/fsm_q CK ) ;
34 END NETS
35
36 END DESIGN

```

Note three things. First, every component line carries its **reference cell name** (so the tool can look the geometry up in LEF), its **placement status** (FIXED vs. PLACED vs. UNPLACED), and its **orientation** (N, S, FN, etc.). Second, the file lists a count after each section header (COMPONENTS 4 ;) so the parser can pre-allocate. Third, all coordinates are integers in DEF database units, not microns.

Gut check

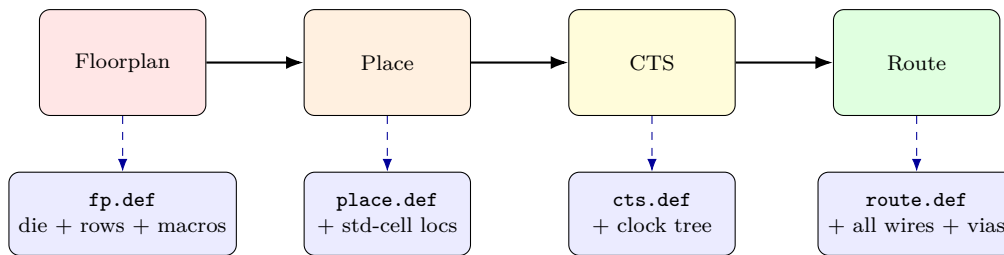
In Listing 7.1, which two instances are **fixed** and cannot be moved by the placer, and which can?

ENDCAP_L_0 is FIXED (physical-only end-cap cell, locked at floorplan time). The three logical instances (u_alu/g_carry, u_alu/g_sum, u_ctrl/fsm_q) are PLACED, meaning the placer chose locations but is allowed to refine them in subsequent stages unless promoted to FIXED.

7.4 DEF Lives at Every PD Checkpoint

This is the part most beginners miss. DEF is not a one-shot file. The **same format** is emitted by the tool after each major stage of the PD flow, with progressively more sections populated. The floorplan-stage DEF has rows and macros but empty **NETS** geometry. The post-route DEF has every wire segment and every via.

This is why “DEF file” on its own is ambiguous in conversation. A senior PD engineer will ask *which* DEF: the post-place one or the post-route one. They are very different in size and in what they let you do.



Same DEF grammar at each stage. Each downstream tool reads the prior DEF as input.

Figure 7.2: DEF is both an **output** of every PD stage and the **input** to the next. The grammar never changes; only the populated sections grow. **NETS** routing geometry is empty until after detailed routing.

Memory hook

Think of DEF like the **photo a contractor texts you each Friday**. Week 1 is just the foundation slab (floorplan). Week 5 is framing (placement). Week 9 has wiring (route). Same camera, same angle, more stuff in every shot. Skip a week's photo and the next subcontractor shows up to a job site they cannot read.

Pause

Pause. You inherit a directory with three DEFs named **a.def**, **b.def**, and **c.def** and no documentation. Which one diff would tell you which file came from which stage **without opening a layout viewer**? What grep would you reach for first?

7.5 Worked Example: Coordinate Arithmetic

DEF coordinates are integers, not microns. The conversion factor lives in the **UNITS** statement at the top of the file.

Problem. The DEF in Listing 7.1 starts with **UNITS DISTANCE MICRONS 1000**. An instance is written as **PLACED (12500 7500) N**. What is its physical placement in microns, and what is the smallest distance the format can represent?

Move 1. Read the scale. **UNITS DISTANCE MICRONS 1000** means **1 micron equals 1000 DEF database units**. So 1 DEF unit equals $1\mu\text{m}/1000 = 1\text{ nm}$.

Move 2. Convert the X coordinate. $12500\text{ DEF units} \times \frac{1\mu\text{m}}{1000\text{ units}} = 12.5\mu\text{m}$.

Move 3. Convert the Y coordinate. $7500/1000 = 7.5\mu\text{m}$.

Move 4. Resolution. The smallest non-zero distance representable is **1 nm**, since coordinates must be integers in DEF units.

Move 5. What changes at a finer node. For a 7 nm flow you often see **UNITS DISTANCE MICRONS 2000**, meaning 1 DEF unit = 0.5 nm. The same coordinate string 12500 now means $12500/2000 = 6.25\mu\text{m}$, which is why you **cannot copy coordinates between DEFs with different UNITS scales** without rescaling.

Gut check

A DEF declares `UNITS DISTANCE MICRONS 2000` and pins one IO at (36000 18000). Where does that pin physically sit?

$36000/2000 = 18\ \mu\text{m}$ in *X* and $18000/2000 = 9\ \mu\text{m}$ in *Y*. Note that if you had assumed the previous file's 1000 scale, you would have placed it at (36, 18) and been off by 2× in both axes, which is a classic ECO bug.

Watch Out

DEF traps that bite new PD engineers.

- **DEF vs. LEF confusion.** LEF is the *library* physical view (what a cell looks like, reusable across designs). DEF is the *design* snapshot (where every instance of every cell sits in *this* chip). LEF is mandatory to start PD; DEF is not, because the floorplan tool can generate the first DEF itself.
- **Coordinate-unit mismatch.** Forgetting to read `UNITS DISTANCE MICRONS` before doing arithmetic on DEF coordinates is the single most common rookie error. Always grep the header first.
- **DEF is both output and input.** The same file role flips at every stage boundary. Post-route DEF is an output of detailed routing **and** the input to signoff extraction, formal LVS, and any later ECO. Treating it as one or the other (but not both) leads to broken handoff scripts.
- **DEF is not GDS.** Tape-out goes out as **GDSII** (binary, polygon-level mask data). DEF stays inside the PD environment. You never hand DEF to the foundry as final artwork.

Sidebar: optional depth, skip if tired

DEF as a design-exchange artifact. The name is not a coincidence. DEF was invented so that two teams (or two companies) could hand a partial physical implementation back and forth without sharing the whole tool database. A vendor can deliver a hardened block as **LEF + post-route DEF**; the customer imports it into their top-level integration without ever seeing the source RTL or the cell internals.

DEF in ECO flows. Engineering Change Orders work the same way. The signoff team writes a tiny **ECO patch DEF** that touches only the instances and nets that changed, and the implementation tool reads it as an incremental input over the existing routed DEF. Same format, smaller delta, much faster iteration than redoing place-and-route from scratch.

Why PD Cares

Why PD cares.

- **ARM PD intern lens:** on your first day, you will be asked to load *.def into Innovus or ICC2 and screenshot the floorplan; knowing which DEF (floorplan vs. post-route) you are looking at is half of not embarrassing yourself in the standup.
- **Generic foundry-flow PD engineer lens:** every stage script writes a DEF and the next stage reads it; broken handoffs almost always trace to a `UNITS` mismatch, a missing `PINS` section, or a stale post-place DEF being fed into routing instead of the post-CTS version.

- **VLSI interview lens:** expect “what is the difference between LEF and DEF?” and “at what stages is a DEF file generated?”; the right answers are *library vs. design instance* and *after floorplan, place, CTS, and route*.

Checkpoint

You should now be able to: (1) state that DEF is an ASCII snapshot of the design’s physical state and name its main sections (**DIEAREA**, **ROW**, **COMPONENTS**, **PINS**, **NETS**), (2) explain why DEF is emitted at every PD checkpoint and how the same format grows in completeness across stages, (3) convert DEF database units to microns using the **UNITS DISTANCE MICRONS** scale, and (4) cleanly separate LEF (library, abstract cell view) from DEF (this design’s instance placement) and from GDS (final mask artwork for the fab). **Say it in 2 seconds: DRCPN: DIEAREA, ROW, COMPONENTS, PINS, NETS**

Chapter 8

The CTS Spec: How You Tell the Tool to Build the Clock

TL;DR

- **Last chapter:** DEF was the snapshot of where everything lives. **This chapter:** the CTS spec is the instructions for how the biggest single set of those things (the clock network) gets built between post-place and post-route DEF.
- The **CCOPT spec** is a Tcl/CSV artifact that tells Innovus the per-clock targets: **skew, latency, max transition, NDR, skew groups, useful-skew limits**, and mesh boundaries.
- Clock nets do not route like data nets. They get **non-default routing rules (NDR)** for shielding and width, and they get **balanced** across endpoints inside a skew group.
- Get the spec wrong and you spend a week chasing a clock spine that ate three M5 tracks too many or a generated clock that drifted 60 ps from its master.

Memory hook

Open post-place DEF and post-route DEF side by side. Same netlist, you would think, with wires filled in. Except the cell count grew by roughly five percent, every flop has a new buffer chain feeding it, and the clock nets look nothing like what synthesis handed you. Something ran in between that cloned hundreds of buffers, picked routing layers the data nets are not allowed to touch, and balanced arrival times to within ten picoseconds at thousands of sinks. What did that thing read to know what to do?

8.1 What the CTS Spec Actually Is

The **CTS spec** (in Innovus, the **CCOPT spec**) is a small text file (Tcl directives plus an optional CSV) that tells `cchopt_design` how to build and balance the clock network. It is not RTL, it is not constraints in the SDC sense, and it is not floorplan. It is a **per-clock recipe**: for each clock root, what is the target latency, what is the target skew, which other clocks share its skew budget, what routing rule do the segments use, and where (if anywhere) does the network transition from a tree to a mesh.

The Synopsys ICC2 equivalent lives inside `clock_opt` and a `.ccopt`-equivalent CSV; the vocabulary is nearly identical, the keywords differ.

Key Idea

The CTS spec is the **only place in the flow where you express clock-network intent declaratively**; everything else (placement, route, sign-off) just reacts to the tree CCOPT builds from it.

A realistic CCOPT spec snippet looks like this:

Listing 8.1: Excerpt from a CCOPT spec for a two-clock subsystem.

```

1 # ---- global CTS knobs ----
2 set_ccopt_property buffer_cells {CKBUFM8 CKBUFM12 CKBUFM16}
3 set_ccopt_property inverter_cells {CKINVM8 CKINVM12 CKINVM16}
4 set_ccopt_property clock_gating_cells {CKLNQM8 CKLNQM12}
5 set_ccopt_property target_max_trans 80ps
6 set_ccopt_property use_inverters true
7
8 # ---- per-clock targets ----
9 create_ccopt_clock_tree clk_core -source [get_db pins CLKGEN/clk_core]
10 set_ccopt_property -clock_tree clk_core target_skew 50ps
11 set_ccopt_property -clock_tree clk_core target_latency 350ps
12 set_ccopt_property -clock_tree clk_core routing_rule NDR_2W2S_M5M6
13
14 create_ccopt_clock_tree clk_io -source [get_db pins CLKGEN/clk_io]
15 set_ccopt_property -clock_tree clk_io target_skew 75ps
16 set_ccopt_property -clock_tree clk_io routing_rule NDR_1W2S_M4M5
17
18 # ---- skew groups ----
19 create_ccopt_skew_group -name core_grp \
20     -sources {CLKGEN/clk_core CLKGEN/clk_core_div2}
21 create_ccopt_skew_group -name io_grp -sources {CLKGEN/clk_io}
22 create_ccopt_skew_group -name dft_grp -sources {CLKGEN/clk_dft}
23
24 # ---- run it ----
25 ccopt_design -cts

```

Four things are happening here. The first block sets the **cell palette** CCOPT is allowed to clone from. The second creates a tree per clock root with its own skew and latency targets. The third declares **which clocks share a balance budget**. The fourth runs the engine.

Memory hook

A CCOPT spec is the **conductor's score** for the orchestra of flops. Skip the part that says “violins and violas share a downbeat” and the section drifts a quarter note; in silicon a quarter note is 60 ps of skew and your launch-capture pair misses by an entire setup margin.

8.2 Clock Skew Groups

Skew is the difference in clock arrival time between two endpoints. CCOPT will spend buffers and routing to flatten skew, but only *within a skew group*. Two clocks in different groups are allowed to drift relative to each other because the tool assumes you do not have any path that closes timing between them.

You put clocks in the same skew group when **paths exist between their endpoints** (synchronous CDC, generated clocks of a common master, mux-selected clocks that converge on the same flops). You keep them in separate groups when the domains are **truly asynchronous**

(handled by a CDC synchronizer) so CCOPT does not waste buffers trying to flatten a difference no timing path cares about.

Gut check

You have `clk_core` at 800 MHz and `clk_core_div2` at 400 MHz, generated from `clk_core` by a divide-by-two flop. A handful of paths cross from div2 flops back to core flops. Same group or different?

Same group. The two clocks share endpoints, and you need their insertion delays balanced so the cross-domain paths close at sign-off. If you leave the divider in its own group, CCOPT balances the div2 tree to itself and the divider output drifts 40–80 ps from the core tree, blowing the cross-domain setup checks even though the RTL is perfectly synchronous.

Worked numbers: three clocks, two groups

Take three clocks on a small subsystem:

Clock	Frequency	Endpoints
<code>clk_core</code>	800 MHz	CPU pipeline flops
<code>clk_core_div2</code>	400 MHz	Cache tag array (same CPU)
<code>clk_io</code>	400 MHz	PHY-facing flops (async to core)
<code>clk_dft</code>	100 MHz	Scan chain only

Group `clk_core` and `clk_core_div2` together because cache-tag-to-pipeline paths exist. Leave `clk_io` on its own (CDC synchronizer at the boundary). Leave `clk_dft` on its own (only active in scan mode, never racing functional paths).

Slack benefit of grouping the two core clocks: if each tree is built independently CCOPT meets a 50 ps intra-tree skew target but **intra-tree alignment between the two trees drifts by typically 60–100 ps**. Group them and that drift collapses into the same 50 ps target, recovering roughly 60 ps of slack on every cross-clock path. On an 800 MHz clock (1.25 ns period) that is almost five percent of the budget handed back for free.

Pause

Stop here. Write down the rule of thumb in your own words: *when do two clocks belong in the same skew group?* If the answer in your head is anything other than “when timing paths exist between their endpoints,” re-read the last two paragraphs before continuing. The rest of this chapter assumes you have this reflex.

8.3 NDR: Non-Default Routing Rules

Clock nets are skew-sensitive, which means they are crosstalk-sensitive, which means they get **routed differently from data nets**. A **Non-Default Routing rule (NDR)** bumps wire width, wire spacing, or both, and optionally adds **shield wires** tied to power or ground on either side.

The shorthand **2W2S** means double width and double spacing relative to the default for that layer. **1W2S** means default width, double spacing. Width fights resistance and reduces RC delay variation; spacing fights coupling capacitance to neighbors and reduces aggressor-induced skew.

Worked numbers: 2W2S on M5

Assume M5 has **default width 0.07 μm** and **default spacing 0.07 μm** , so default pitch (center to center of adjacent tracks) is $0.07 + 0.07 = 0.14 \mu\text{m}$.

Move 1. A 2W2S NDR sets effective width to $2 \times 0.07 = 0.14 \mu\text{m}$ and effective spacing to $2 \times 0.07 = 0.14 \mu\text{m}$. Pitch between two NDR clock wires becomes $0.14 + 0.14 = 0.28 \mu\text{m}$. **Each NDR clock wire consumes the area of two default tracks plus the gap**, so the effective track footprint is $0.28 \mu\text{m}$ versus $0.14 \mu\text{m}$ for a data net. One clock route eats **two default M5 tracks**.

Move 2. If the spec demands shielding (one VSS wire on each side at default spacing), tack on another $2 \times 0.14 \mu\text{m}$. Total footprint per shielded clock wire: $0.28 + 0.28 = 0.56 \mu\text{m}$, or **four default M5 tracks per clock net**.

Move 3. A clock spine of 8 parallel routes therefore consumes $8 \times 4 = 32$ M5 tracks. On a $100 \mu\text{m}$ wide channel with M5 pitch $0.14 \mu\text{m}$, the channel has roughly 714 tracks total, so the spine eats **about 4.5% of M5 in that channel**, gone before a single data net is routed.

That is the tradeoff you signed up for when you put `NDR_2W2S_M5M6` in the spec. The clock is quiet; the routing congestion picture for everything else got worse.

Watch Out

NDR enforced after routing is the classic disaster. If the routing rule is missing from the CCOPT spec but added later as a post-CTS ECO, the tool either ignores the clock segments already routed or rips them up and reroutes through whatever tracks are still free. On Duc-scale blocks that means a 3-hour route iteration to recover from a one-line spec omission. **Put the NDR in the spec before `ccopt_design`, not after.**

8.4 Useful Skew

Useful skew is the deliberate trick of *not* balancing a clock perfectly. By pushing the capture clock edge a little later at a slow flop, you give the launching path more time to settle (setup relief), at the cost of less time before the next launch fires (hold gets tighter on the downstream stage).

Worked numbers: borrowing 30 ps

Take a flop-to-flop path with:

- **Setup slack:** +50 ps (barely making it)
- **Hold slack:** +80 ps (comfortable)

Push **30 ps of positive skew into the capture flop** (delay its clock by 30 ps relative to the launch flop).

- New setup slack: $50 + 30 = +80$ ps. The capture edge is later, so the data has 30 ps more time to arrive.
- New hold slack: $80 - 30 = +50$ ps. The capture edge is later, so the new launch from this flop has 30 ps less margin before being captured downstream.

The total slack budget did not change. You moved 30 ps from the hold column to the setup column on this stage, and **the next stage downstream now sees its own setup get 30 ps tighter**. Useful skew is a zero-sum trade across pipeline stages; CCOPT does the bookkeeping when you give it `set_ccopt_property -clock_tree <clk> useful_skew true` along with a per-pin limit like ± 50 ps.

Gut check

You enable useful skew with a limit of ± 100 ps on a tight CPU. Setup violations clear up. Two days later, hold violations explode and a chunk of them are unfixable without buffering the data path so heavily it re-breaks setup. What happened?

The limit was too aggressive for the pipeline depth. Useful skew shifted setup margin into hold on every borrowed path, and adjacent stages compounded the shift. Drop the limit to ± 30 or ± 40 ps, and reserve the larger window for a hand-picked list of critical endpoints rather than every flop in the block.

8.5 Clock Mesh vs Clock Tree

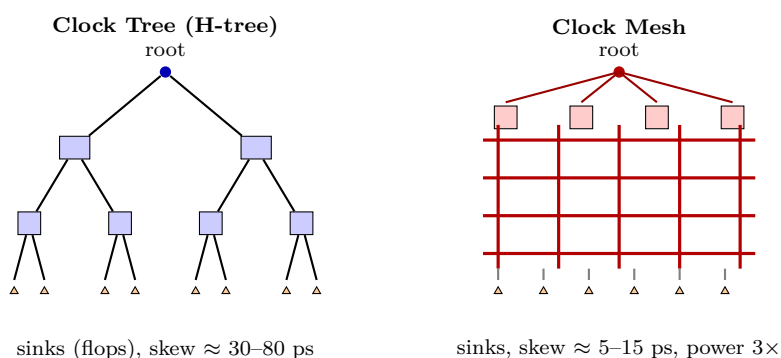


Figure 8.1: An H-tree (left) drives sinks through a fanout hierarchy of buffers; skew is moderate, power is moderate. A clock mesh (right) shorts together many parallel drivers across a metal grid so every sink sees nearly the same arrival time; skew is tiny, power and area are large.

A **clock tree** (H-tree, fishbone, or balanced binary tree) drives sinks through a hierarchy of cloned buffers. CCOPT picks insertion points, sizes drivers, and balances branches. This is the default and it is what 95% of blocks ship with.

A **clock mesh** shorts the outputs of many parallel drivers onto a metal grid, so every sink connects to a low-impedance plane that arrives at nearly the same time everywhere. Skew drops to single-digit picoseconds. **Power burns roughly $2\text{--}3\times$** a comparable tree, because the grid is constantly switching and the parallel drivers all fight each other slightly. Mesh wins on multi-GHz CPU cores where 5 ps of skew determines the frequency the part hits. Tree wins on everything else.

Mesh is configured through a different family of CCOPT directives (mesh region, driver list, post-mesh tuning) and we will not unpack those here; the point is that the **same spec file** is the entry point for both worlds.

Sidebar: optional depth, skip if tired

What ccopt_design actually does under the hood. It is one Tcl command but the most computationally expensive single step in the flow, often hours on a large block. Internally it runs a sequence of passes: (1) *clone* clock gates and buffers so fanout is balanced; (2) *restructure* the logical hierarchy of the clock so latencies match across endpoints in the same skew group; (3) *insert and size buffers* along the now-balanced topology; (4) *legalize* placement after the inserted cells push neighbors around; (5) *route* the clock nets with the declared NDR; (6) *post-CTS optimize* data paths now that real clock latencies exist. Every one of those passes reads the CCOPT spec. Change one line in the spec and all six passes redo their work.

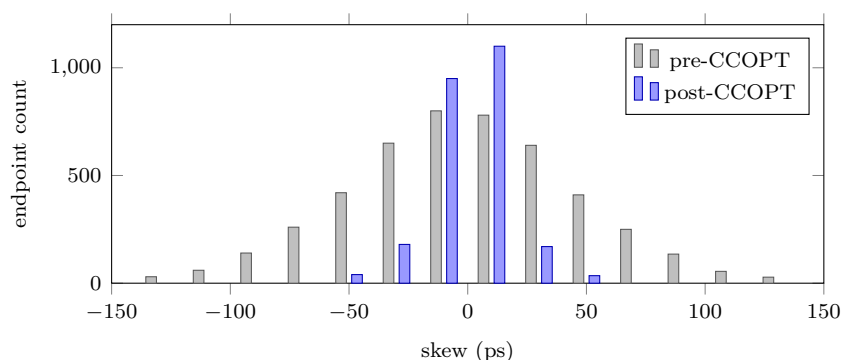


Figure 8.2: Endpoint skew distribution before and after `ccopt_design` on a representative block. Pre-CTS the skew spread is essentially the placement-derived latency mismatch (often ± 130 ps). Post-CCOPT the distribution collapses to within the spec'd target skew (here 50 ps).

Why PD Cares

Why PD Cares

- **Arm Duc, your job right now.** Duc CTS work means you are reading CCOPT logs every day; understanding NDR enforcement order is the difference between debugging a clock spine in 20 minutes vs 2 hours. Same for skew groups: a generated clock dropped from the wrong group will produce cross-domain timing failures that look like SDC bugs and are actually a one-line CCOPT spec fix.
- **General PD engineer.** CTS is the only step where the tool builds new logic (clones buffers, inverters, gates) without you naming them in RTL. The spec is the contract that governs that creation; everything you do upstream (placement, floorplan) and downstream (route, sign-off) is reacting to what CCOPT decided.
- **VLSI interview.** “What is useful skew, and when is it dangerous?” is asked in roughly half of physical-design interviews. So is “how do skew groups interact with generated clocks?” Memorize the worked examples in this chapter and you will outscore most candidates.

Checkpoint

You should now be able to (a) open a CCOPT spec and explain every `set_ccopt_property` line, (b) decide which clocks share a skew group and which do not, (c) compute the M5 track cost of a 2W2S NDR, (d) trade setup against hold with useful skew on a numerical path, and (e) know when mesh is worth its power bill.

Say it in 2 seconds: *Spec the skew, shield the spine, borrow the slack.*

Chapter 9

Parasitic Files: ITF, the Mapping File, and SPEF

TL;DR

- **Callback to Chapter 8 (CTS Spec):** Last chapter, the CTS spec told the tool how to build the clock. This chapter, parasitic files tell the tool (and the STA engine) what every wire it just built *actually* does electrically.
- The **ITF** is the foundry's description of the metal stack: thicknesses, resistivities, dielectric constants. It is the source of truth for everything downstream.
- The **mapping file** bridges LEF/DEF layer names (M3, VIA12) to ITF layer indices. One typo here silently corrupts all RC.
- **SPEF** is the per net RC tree the extractor emits. Half of signoff timing debug is reading SPEF by hand.

Memory hook

The router placed an 80 μm wire on M3 between two flops. STA reports 47 ps of delay on that net. Where did the 47 come from? Not from the cell. Not from the SDC. It came from a chain of files most engineers never open: a foundry ITF, a tiny mapping file, and a SPEF that the extractor cooked up at 3am during signoff. This chapter cracks that chain open.

9.1 ITF: The Interconnect Technology File

The **ITF** (Interconnect Technology File) is a foundry deliverable that describes the back end of line stack exactly as the extraction engine sees it. Every metal layer has a thickness, a minimum width, a sheet resistance, and an etch profile. Every dielectric layer between metals has a thickness and a relative permittivity ϵ_r . Nothing in your timing comes out right if the ITF is wrong.

Key Idea

The ITF is not read by Innovus or PrimeTime directly. It is compiled by the foundry's extraction vendor (Synopsys StarRC, Cadence QRC) into a binary **TLU+** file or **QRC techfile**. That binary is what the extractor actually loads. The ITF is the human readable source; the TLU+ is the artifact your flow consumes.

A small excerpt looks like this:

Listing 9.1: ITF excerpt for a 7nm-ish stack.

```

1 TECHNOLOGY = my_7nm_stack
2 GLOBAL_TEMPERATURE = 25
3
4 CONDUCTOR M3 {
5     THICKNESS = 0.16
6     WMIN = 0.07
7     SMIN = 0.07
8     RPSQ = 0.072
9     ETCH_VS_WIDTH { VALUES = (0.07, 0.005) (0.20, 0.008) }
10 }
11
12 DIELECTRIC ILD3 {
13     THICKNESS = 0.20
14     ER = 2.7
15 }
16
17 DIELECTRIC ILD3_CAP {
18     THICKNESS = 0.05
19     ER = 4.1 # nitride cap layer
20 }
```

Read it top down. M3 is 0.16 μm thick, the narrowest wire allowed is 70 nm, the sheet resistance is 0.072 $\Omega/\square$, and the etch shrinks the drawn width by a width dependent amount. Below M3 sits a 200 nm ILD with $\epsilon_r = 2.7$ (a low-k oxide), capped by a thin nitride layer at $\epsilon_r = 4.1$. The extractor combines all of this with the actual routed geometry to produce R and C per segment.

Memory hook

The ITF is the stack's birth certificate The **foundry ships** an ITF that **freezes** the metal stack's electrical truth. Lose it, mis version it, or load the wrong corner, and **every SPEF downstream is fiction**.

Gut check

You see a TLU+ file in your flow but no ITF anywhere in the release. Is that normal?

Yes. Foundries often ship only the compiled TLU+ (and sometimes a redacted ITF) because the full ITF reveals proprietary process details. You consume the TLU+; you trust the foundry compiled it from the right ITF.

9.2 The Mapping File: Tiny, Mission Critical

The router speaks LEF/DEF layer names: M1, M2, VIA12. The ITF speaks its own layer indices and internal names. The **mapping file** (sometimes called the **nextgrd** map, the **layer_map**, or the **tech2itf** map) is the one line per layer dictionary that joins them.

Listing 9.2: A tiny but mission critical mapping file.

```

1 # LEF/DEF name ITF name type
2 M1 METAL1 routing
3 M2 METAL2 routing
4 M3 METAL3 routing
5 VIA12 VIA1 via
6 VIA23 VIA2 via
```

If M3 in the LEF is wrongly mapped to METAL2 in the ITF, the extractor happily uses the thinner METAL2 thickness for every M3 segment in your design. Resistance is wrong by a constant factor. Capacitance is wrong by a different constant factor. Your STA looks clean. Silicon comes back broken.

Watch Out

A missing mapping line does *not* always error out. Some flows fall back to a default layer or skip extraction on that layer entirely. You will see a warning buried in the extractor log and a SPEF that is missing parasitics on, say, every M7 net. Always grep the extraction log for **WARN** and **unmapped**.

9.3 SPEF: Where Parasitics Become a File You Read

SPEF (Standard Parasitic Exchange Format) is the IEEE 1481 text format the extractor writes once per net. It contains the routed net's RC tree: every pin, every internal node (where a wire bends or branches), every R segment between nodes, and every C to ground (or coupling C to neighbor nets).

Move 1. Set up the net. Imagine a net **n42** driven by **U1/Y**, fanning out to two flop D pins, **FF1/D** and **FF2/D**. The router put down 80 μm of M3 total: 40 μm from the driver to a branch point, then 20 μm to each sink.

Move 2. Compute the resistances. M3 RPSQ is $0.072 \Omega/\square$. Wire width is 70 nm, so squares per micron = $1/0.07 \approx 14.3$. R per micron = $14.3 \times 0.072 \approx 1.03 \Omega/\mu\text{m}$. So the 40 μm driver segment is $\sim 41 \Omega$ and each 20 μm sink segment is $\sim 21 \Omega$.

Move 3. Compute the capacitances. At 7nm M3 a sane total C is roughly $0.18 \text{ fF}/\mu\text{m}$ (ground + coupling lumped). 40 $\mu\text{m} \rightarrow 7.2 \text{ fF}$; 20 $\mu\text{m} \rightarrow 3.6 \text{ fF}$. Split each segment's C into two lumps at its endpoints (the π model the extractor uses).

Move 4. Lay out the nodes. Call them 1 (driver pin), 2 (branch), 3 (FF1/D), 4 (FF2/D). Caps lump at nodes 1, 2, 3, 4. Resistors sit between 1–2, 2–3, 2–4.

Move 5. Pick the cap totals at each node. Driver pin node 1 gets $\sim 3.6 \text{ fF}$ from half of the driver segment. Branch node 2 gets the other half of the driver segment plus half of each sink segment: $3.6 + 1.8 + 1.8 = 7.2 \text{ fF}$. Sink nodes 3 and 4 each get the other half of their sink segment: $\sim 1.8 \text{ fF}$ each, plus the pin cap of the receiving flop ($\sim 1.5 \text{ fF}$), total $\sim 3.3 \text{ fF}$.

Move 6. Write it as SPEF. See Listing 9.3.

Move 7. Sanity check the totals. Total wire C: $3.6 + 7.2 + 3.3 + 3.3 = 17.4 \text{ fF}$, close to the $0.18 \times 80 = 14.4 \text{ fF}$ wire estimate plus $2 \times 1.5 \text{ fF}$ pin caps. Total R from driver to FF1: $41 + 21 = 62 \Omega$. Numbers feel right for 7nm M3.

Listing 9.3: SPEF snippet for net **n42**. Reduced and slightly idealized.

```

1 *SPEF "IEEE 1481-1999"
2 *DESIGN "duc_top"
3 *DATE "Mon May 26 14:02:11 2026"
4 *VENDOR "Cadence"
5 *PROGRAM "Quantus"
6 *VERSION "21.1"

```

```

7 *DESIGN_FLOW "PIN_CAP NONE" "FULL_CONNECTIVITY"
8 *DIVIDER /
9 *DELIMITER :
10 *BUS_DELIMITER [ ]
11 *T_UNIT 1 PS
12 *C_UNIT 1 FF
13 *R_UNIT 1 OHM
14 *L_UNIT 1 HENRY
15
16 *D_NET n42 17.4
17 *CONN
18 *I U1:Y 0 *L 0 *D BUFX4
19 *I FF1:D I *L 1.5 *D DFFX1
20 *I FF2:D I *L 1.5 *D DFFX1
21 *CAP
22 1 n42:1 3.6
23 2 n42:2 7.2
24 3 n42:3 3.3
25 4 n42:4 3.3
26 *RES
27 1 n42:1 n42:2 41.0
28 2 n42:2 n42:3 21.0
29 3 n42:2 n42:4 21.0
30 *END

```

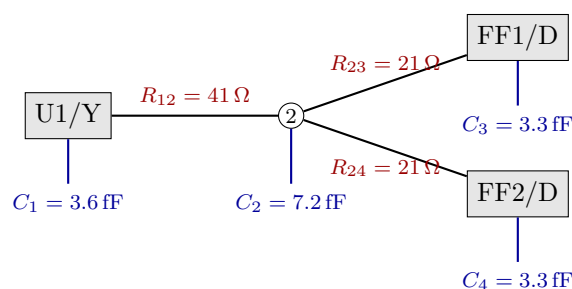


Figure 9.1: RC tree for net `n42` reconstructed from the SPEF in Listing 9.3. Caps shown as lumps to ground; resistors between nodes.

Pause

Before you read on, stare at Listing 9.3 and Figure 9.1 side by side. The `*CAP` section is just a list of (node, value) pairs. The `*RES` section is just (from, to, value). Once that clicks, SPEF stops being scary and becomes a circuit netlist you can sketch on paper in 60 seconds.

9.4 How STA Actually Uses SPEF

The STA engine does *not* simulate the RC tree as a SPICE deck (way too slow for millions of nets). It reduces the tree to an **effective capacitance** C_{eff} seen by the driver and an **Elmore delay** (or higher order moment) to each sink. CCS and ECSM models then use C_{eff} to pick the right driver waveform from the `.lib`.

The **Elmore delay** to a sink is the sum, over every resistor R_i on the path from driver to sink, of R_i times the total downstream capacitance hanging off R_i .

Memory hook

Elmore = R times all the C it sees downstream For each **resistor on the path, multiply it by every cap downstream** of it, **sum the products**. Skip a downstream branch and your delay is wrong by the branch's contribution.

Worked example for the path U1/Y → FF1/D in Figure 9.1:

- $R_{12} = 41\ \Omega$ sees everything downstream of node 2: $C_2 + C_3 + C_4 = 7.2 + 3.3 + 3.3 = 13.8\ \text{fF}$.
- $R_{23} = 21\ \Omega$ sees only $C_3 = 3.3\ \text{fF}$.

$$t_{\text{Elmore}} = 41 \times 13.8 + 21 \times 3.3 = 565.8 + 69.3 = 635.1\ \Omega \cdot \text{fF} = 0.635\ \text{ps}.$$

That is the *wire delay* contribution at sink FF1/D. The driver's intrinsic delay (from the .lib, evaluated at the effective load) is added on top. The 47 ps from the chapter opener is mostly the driver, with a small contribution from this Elmore term. Now you know where it came from.

Gut check

If the router rerouted node 2 farther from the driver, doubling R_{12} to $82\ \Omega$, what happens to FF1's Elmore delay?

*The first term doubles: $82 \times 13.8 = 1131.6$. Total becomes $1131.6 + 69.3 = 1200.9\ \Omega \cdot \text{fF} \approx 1.2\ \text{ps}$.
The shared resistor near the driver dominates because it sees the entire downstream capacitance.*

9.5 SPEF Flavors and Corners

SPEF is not a single file. The extractor emits one SPEF *per RC corner*, and modern signoff loads them all under MMMC. The standard set is:

- **Cworst**: worst case coupling cap. Used for setup analysis when long wires hurt the most.
- **Cbest**: best case (lowest) cap. Used for hold analysis where fast nets cause race conditions.
- **RCworst**: worst R *and* worst C. Pessimistic for setup on resistive paths.
- **RCbest**: best R and best C. Optimistic; hold analysis on long wires.
- **Typical**: nominal corner for sanity and reporting.

Five SPEFs per scenario. Multiply by your PVT corners, modes, and OCV settings, and the MMMC explosion from the back matter chapter stops being abstract: at Arm signoff you can easily be loading 40+ SPEFs in a single `create_analysis_view` grid.

Watch Out

Comparing the C of a net across two SPEFs and concluding "extraction is broken" is a classic rookie mistake. Check the header. If one is `*DESIGN_FLOW "..."` Cworst and the other is Cbest, they are *supposed* to differ by 30 to 50 percent. They are not the same file.

Sidebar: optional depth, skip if tired

DSPF vs SPEF vs SBPF **DSPF** (Detailed Standard Parasitic Format) is the older, Synopsys originated format. It preserves every internal node with explicit names like

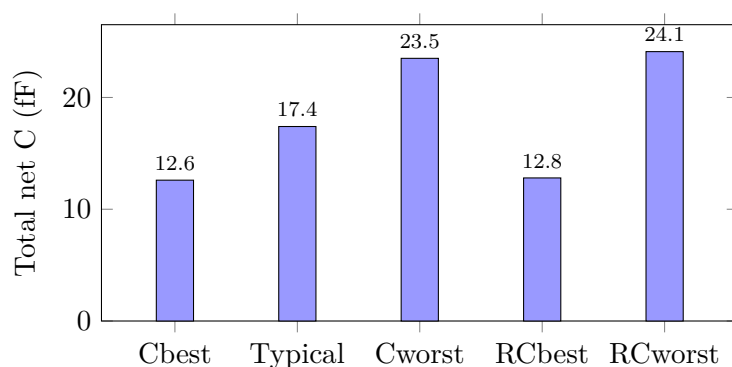


Figure 9.2: Extracted total C for net `n42` across the five RC corners. The same net, the same routing, five different SPEFs.

`n42:1234`, making it easy to map back to physical geometry. SPICE simulators love it. **SBPF** (Synopsys Binary Parasitic Format) is a compressed binary form for huge designs. **SPEF** won as the lingua franca because IEEE standardized it, every vendor reads it, and it supports name mapping (`*NAME_MAP`) to compress the file. Some analog teams still ship DSPF for block level SPICE sims, then the digital flow converts to SPEF for STA. At Arm you will mostly see SPEF; DSPF shows up only when a memory or analog macro hands you their extracted view.

Why PD Cares

Why PD Cares about Parasitic Files

- **At Arm on Duc:** you will be staring at SPEF when a path mysteriously misses timing at signoff that was clean post route. Learning to `grep *D_NET <netname>` out of a 4 GB SPEF, sketch the RC tree, and hand compute Elmore is a core Innovus debugging skill. The first time a senior engineer asks "what's the `Reff` on the second branch?" you want to know exactly which lines to look at.
- **Generic PD engineer:** parasitic correlation between extractor outputs and silicon is one of the top three reasons a tapeout slips. If your team upgrades TLU+ mid project, every block's timing changes and you have to re-close. Understanding what an ITF, mapping file, and SPEF *are* is the prerequisite to even having that conversation.
- **VLSI interview:** "Walk me through how a wire becomes a delay number in STA" is a standard question. The answer (`geometry` → `ITF` → `TLU+` → `extractor` → `SPEF` → `reduction` → `Elmore` → `.lib` lookup) is what this chapter is about. Saying "the tool just figures it out" is a fast way to end the interview.

Checkpoint

Say it in 2 seconds: *ITF cooks the stack, the map joins the names, SPEF tells STA what the wire really is.*

You should now be able to:

- Explain why the ITF lives upstream of TLU+ and never goes into Innovus directly.
- Read a five line mapping file and name what breaks if a line is missing.

- Sketch an RC tree from a *D_NET block in a SPEF in under a minute.
- Compute Elmore delay to any sink given the tree.
- Name the five standard RC corners and which analysis each one feeds.

Chapter 10

What Changes in the Real World

TL;DR

- **Last chapter:** parasitic files (ITF, mapping, SPEF) told the tool what each routed wire actually does electrically. **This chapter:** we zoom out and see how every input covered so far explodes in production: MMMC view counts, pin access, derate models, and DEF size growth through the flow.
- **MMMC is multiplication, not addition.** Five cell families times three voltages times three temperatures times six process corners times four modes is not a clever rhetorical flourish; it is a literal `.lib` count north of one thousand.
- **Pin access is the new floorplan.** At 7 nm, an instance can be DRC-clean in placement and still strand the router because the M1 pin geometry offers too few via-cut candidates.
- **OCV, AOCV, POCV change the answer on the same SDC.** A path that meets timing at 1.05 ns with AOCV can fail at 1.10 ns with flat OCV without a single edit to the constraint file.

Memory hook

A junior engineer hands you a netlist, a tech file, an SDC, a LEF, a Liberty, and a DEF. You set up the run. The tool refuses to launch. The error is one line: `ERROR: analysis_view 'func_ss_0p72_125c_setup' references missing delay_corner`. You did not forget a file. You forgot that the toy version of every input you just read about explodes into a matrix of views the moment the block is real. So what does the matrix actually look like, and where does the explosion bite first?

10.1 MMMC: the Liberty view explosion

The toy mental model says “load a `.lib`, load an SDC, run STA.” The production model says “enumerate every physically possible combination of process, voltage, temperature, cell flavor, and operating mode, then promise the foundry your design works at every cross product.” That promise is what **MMMC** (Multi-Mode Multi-Corner) signoff actually encodes.

10.1.1 Counting the Liberty views directly

Walk through it with the numbers in front of you. The arithmetic is boring; the size of the answer is not.

Move 1. Cell families. A modern 7 nm-class standard cell library ships in five threshold flavors: **RVT, LVT, HVT, ULVT, ELVT**. That is **5**.

Move 2. Voltages. Each flavor is characterized at **nominal, low, and overdrive**. That is **3**.

Move 3. Temperatures. Automotive-aware blocks characterize at -40°C , 25°C , 125°C . That is **3**.

Move 4. Process corners. Setup signoff uses **SS, FF, TT, SF, FS**, and hold typically demands a second **FS variant** with a separate skew assumption. That is **6**.

Move 5. Operating modes. The block runs in **functional, scan-shift, scan-capture, and retention** modes. That is **4**.

Move 6. Multiply. $5 \times 3 \times 3 \times 6 \times 4 = 1080$ Liberty views. The chapter’s old “easily exceeds 200” was a comforting understatement.

Move 7. Wall-clock cost. Liberty parsing scales roughly linearly. At **30 seconds per view**, parsing alone is $1080 \times 30 = 32,400$ seconds, or **nine hours** if you do it serially. In practice tools cache and parallelize, but the order of magnitude is real, and every parse hits disk.

Key Idea

The MMMC view count is the **product** of every characterization axis, not the sum. Adding a single new voltage point does not add one view; it multiplies the entire existing matrix by a new factor.

10.1.2 How Innovus actually structures it

Innovus does not consume 1080 raw Liberty files as 1080 independent things. It organizes them into three objects that compose:

- **library_set**: a bundle of `.lib` files that belong together (one PVT corner across all cell families).
- **delay_corner**: a `library_set` plus an RC corner (the QRC tech file) plus an OCV derate spec.
- **constraint_mode**: an SDC file plus the mode it represents (functional, scan, retention).
- **analysis_view**: the actual scenario you sign off, formed by pairing one `constraint_mode` with one `delay_corner`.

The number that matters at runtime is not the 1080 Liberty views; it is the **number of analysis_views you activate**. A typical 7 nm block runs signoff on roughly **20 to 40 active views** simultaneously, picked from the full matrix to cover worst setup, worst hold, worst leakage, and the mode-specific corners. The full 1080 is the universe; the active set is the working slice.

Memory hook

The Liberty matrix is a warehouse, the analysis_view list is the loading dock. If you forget which crates you actually shipped today, signoff signs off on the wrong block and the tapeout slips a month.

Gut check

A block uses 4 cell families, 2 voltages, 2 temperatures, 5 process corners, and 3 modes. How many Liberty views, and how is the *analysis_view* number different from that?

$4 \times 2 \times 2 \times 5 \times 3 = 240$ Liberty views. The *analysis_view* count is whatever subset of (*constraint_mode* $\times$ *delay_corner*) you actually activate for signoff, typically 15 to 30 in production. The matrix sizes the disk; the active list sizes the runtime.

10.2 Pin access: when LEF stops being enough

At 28 nm, a standard cell's LEF PIN section drew an M1 rectangle spanning the cell width. The router saw a long target, dropped a via almost anywhere along it, and moved on. At 7 nm that same geometry strands the router, and placement legality stops implying routability.

10.2.1 Why the 28 nm pin shape fails at 7 nm

Move 1. Track pitch. A 7 nm M1 routing pitch is roughly **36 nm**. The router can only place a wire on one of those tracks; nothing in between is legal.

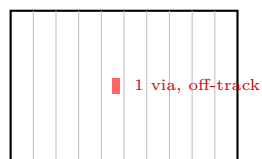
Move 2. Pin geometry must land on a track. A pin rectangle whose center sits between two tracks is invisible to the router. The cell can be placed, but no legal wire can touch it.

Move 3. Via cuts are quantized. A 7 nm V1 cut is roughly 32×32 nm and must obey same-cut spacing. A pin shape that is only **one track wide** offers **one via candidate**. If that candidate clashes with a neighbor cell's via, the router has zero alternatives.

Move 4. Multi-access pins. Modern foundry cell libraries require each input pin to offer at least **two non-conflicting via candidates**, sometimes three. This is enforced by a separate deliverable.

Move 5. Pin access models. The foundry ships a **pin access model** (often a CSV or proprietary table per cell) listing which access points are usable under which neighbor contexts. It augments the LEF; it does not replace it.

Bad pin (28 nm habit at 7 nm)



Good pin (7 nm-aware)

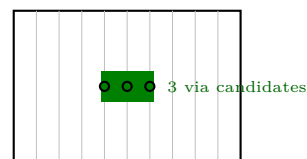


Figure 10.1: Pin access geometry at 7 nm. The left pin is on-grid for placement but off-grid for routing tracks and offers a single via candidate; the router has no fallback when the neighbor cell wants the same cut. The right pin spans three tracks and offers three independent via candidates.

Watch Out

“Placement DRC clean” at 7 nm does **not** mean “routable.” A block can finish placement with zero violations and then accumulate thousands of **unconnected nets** in detail route because of pin-access conflicts the placer never modeled. Always check the pin-access report before signing off on a placement milestone.

10.3 AOCV and POCV: same SDC, different answer

The SDC defines the constraints. The derate model defines how pessimistic the tool is when it applies those constraints. Three derate models dominate production today: **flat OCV**, **AOCV** (Advanced OCV, depth-aware), and **POCV** (Parametric OCV, statistical).

10.3.1 Worked example: a 12-level path at 1.0 ns nominal

The path has **12 logic levels** (12 cells on the data path between launch and capture flops). Nominal cell-plus-net delay sums to **1.0 ns**. The clock period is 1.05 ns.

Move 1. Flat OCV. The launch path is derated up by 1.10, the capture path down by 0.90. Launch delay becomes $1.0 \times 1.10 = 1.10$ ns. With a 1.05 ns period the path **misses by 50 ps**.

Move 2. AOCV table lookup. The AOCV table is depth-dependent: **1.15** at depth 1, **1.05** at depth 12, with linear interpolation between. The longer the path, the less pessimism, because statistical variation of independent stages partially cancels.

Move 3. AOCV applied. At depth 12 the multiplier is **1.05**. Launch delay is $1.0 \times 1.05 = 1.05$ ns. The path **just meets** the 1.05 ns period.

Move 4. POCV per-cell sigma. Each cell publishes a σ/μ ratio in a `.lib` extension. Assume the 12 cells contribute equal mean delay (83.3 ps each) and each has $\sigma/\mu = 0.05$. Per-cell $\sigma = 0.05 \times 83.3 = 4.17$ ps.

Move 5. POCV sqrt-sum-of-squares. Path $\sigma = \sqrt{12 \times 4.17^2} = \sqrt{208.7} = 14.4$ ps. The path mean is 1.000 ns.

Move 6. POCV signoff (mean + 3σ). Launch delay is $1.000 + 3 \times 0.0144 = 1.000 + 0.043 = 1.043$ ns. The path meets with **7 ps of margin**, more than AOCV gave you.

Move 7. Tally.

- Flat OCV: 1.10 ns, **fails by 50 ps**.
- AOCV: 1.05 ns, **meets by 0 ps**.
- POCV: 1.043 ns, **meets by 7 ps**.

Same SDC. Same netlist. Same instance placement. Three different signoff verdicts because the derate engine changed.

Pause

Halfway recap. MMMC says “how many corners.” Pin access says “can the router even reach.” OCV/AOCV/POCV says “how mean is the signoff math.” All three modify outputs the SDC and Liberty already promised, so the toy mental model of “one .lib, one SDC, one answer” is a story the production flow does not tell.

Gut check

The same 12-level path under flat OCV fails by 50 ps; under POCV it passes with 7 ps margin. Did the silicon get faster?

*No. The silicon did not change. The **statistical model of how variation accumulates** changed.*

Flat OCV assumes every stage hits its worst case simultaneously, which becomes wildly pessimistic as path depth grows; POCV models that independent stages partially cancel and reports a tighter, more physically realistic margin.

10.4 DEF growth through the flow, with post-CTS as the surprise

DEF is the same file format at every PD checkpoint, but the byte count tells a story about what each stage added. The post-CTS jump is the one new engineers miss, because the netlist itself changed underneath them.

10.4.1 Worked example: a 1M-instance block

Take a 1 million-instance block and follow the DEF through the flow.

Move 1. Post-floorplan. DEF holds DIEAREA, ROWS, IO pin placement, and macro placement. Standard cells are not placed; COMPONENTS lists them with placeholder status. Size: **~30 MB**.

Move 2. Post-place. Every one of the 1 M instances now has an (x, y) coordinate and an orientation. Each COMPONENT line grows by roughly 40 bytes. Net topology has not been written yet. Size: **~80 MB**.

Move 3. Post-CTS. The CTS engine inserted new instances, roughly **5% of the original count**, as clock buffers, inverters, and ICGs. That is **50 000 new components**. It also **rewrote every clock NET** to reflect the new tree topology. Size: **~120 MB**.

Move 4. Post-route. Every NET now carries full routing geometry: ROUTED segments with layer, coordinates, and via instances. Average net contributes 200 to 400 bytes of geometry. Size: **~350 MB**.

Move 5. The post-CTS diff is the most informative diff in the flow. Run `diff post_place.def post_cts.def`. The only changes you should see are (a) new clock-buffer COMPONENT entries and (b) rewritten clock NET entries. If you see signal nets change, CTS touched something it should not have.

Memory hook

The post-CTS DEF gains 40 MB and 50 000 instances overnight; if you do not diff it against post-place, you will not notice when CTS quietly buffered a signal net and your scan chain breaks at silicon.

Sidebar: optional depth, skip if tired

Why does the post-route jump dwarf everything else? Because routing geometry is the only DEF content that scales with **net length in microns**, not with **instance count**. A single long bus net on M5 can contribute as much DEF text as a hundred short local nets. This is also why `def2stream` for GDS conversion is the slowest step in the back-end: it is parsing geometry, not instances.

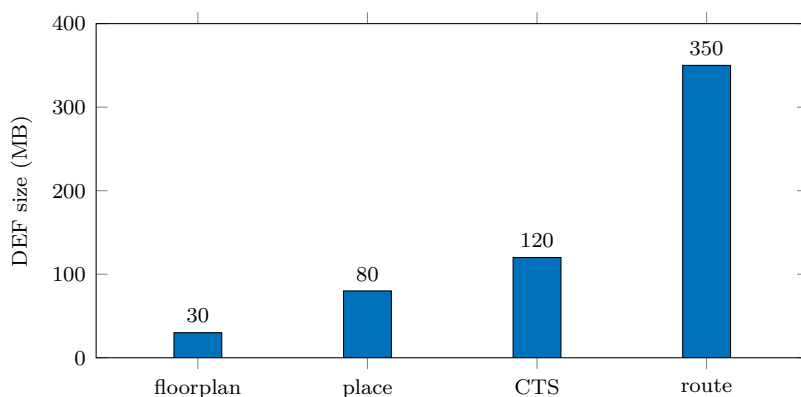


Figure 10.2: DEF size growth across PD stages for a 1 M-instance 7 nm block. The post-place to post-CTS jump (+40 MB) reflects 50 000 inserted clock instances and rewritten clock NETs. The post-route jump (+230 MB) is dominated by per-net routing geometry.

10.5 Why PD Cares

Why PD Cares

Arm PD intern lens (you, this summer on Duc): you will load MMMC views in your first week, and the first error will be a missing `analysis_view` reference in the Innovus setup script. Knowing that `analysis_view = constraint_mode × delay_corner` is what unblocks you in fifteen minutes instead of two days. You will also spend serious time staring at post-CTS DEF diffs because Duc is CTS-heavy.

Generic PD engineer lens: when a tool “runs out of memory” or “hangs on STA setup,” the answer is almost always MMMC scenario explosion or a derate file mismatch, not an algorithmic regression. Check the view count first.

VLSI interview lens: expect “what is the difference between OCV, AOCV, and POCV, and when would you use each?” as a standard signoff-flow question. The right answer leads with the numbers (flat is most pessimistic, POCV is least pessimistic on long paths) and then names the formulas.

Checkpoint

Production PD warps four inputs around the toy mental model: Liberty views **multiply**, pins must **align**, derate **rewrites the verdict**, and DEF **grows fastest at CTS and route**. Say it in 2 seconds: *Multiply, Align, Derate, Grow (MADG)*.

Chapter 11

Common Mistakes

A numbered list, intentionally short. Every one of these is something a PD intern (or interviewer) has actually seen.

1. **Loading the wrong .lib corner** and wondering why the design closes timing at the typical corner but blows up at SS. The .lib loaded is not the corner you think.
2. **Cell name mismatch** between synthesized netlist and the .lib (cell renamed across library versions, or wrong library swapped in). PD throws missing-reference errors before floorplan even starts.
3. **Forgetting to define a generated clock**, so STA reports timing on a phantom unconstrained clock and the slack looks fine when it is silently meaningless.
4. **Using set_false_path as a hammer** to make a violating endpoint go away. The path is still real in silicon; you just stopped checking it.
5. **Pin geometry off the routing grid** in cell LEF. The placer is happy. The router cannot connect.
6. **Confusing tech file with tech-LEF**. They overlap. They are not identical. Tools differ in which they consume.
7. **Comparing DEF coordinates raw** without honoring UNITS DISTANCE MICRONS. A coordinate of 12500 is 12.5 μm at scale 1000 and 6.25 μm at scale 2000.
8. **Skipping the MMMC scenarios at intermediate PD stages** and only running full MMMC at signoff. Late surprises are the most expensive surprises.
9. **Treating LEF and DEF as the same thing**. LEF is library (abstract cell views). DEF is design (this design's instances and their placement).
10. **Editing the netlist by hand** late in the flow to “fix” something. Synthesis owns the netlist. Hand-edits diverge from RTL and create ECO chaos at signoff.

Chapter 12

Interview-Ready 90-Second Answer

The setup. An interviewer says: “Walk me through the inputs a Physical Design tool needs to start a job, and what each one contributes.” You have ninety seconds. Cover the italicized block below with your hand. Recite your version out loud. Then uncover and compare.

*A PD tool needs nine inputs. Three are static descriptions of the parts (**netlist**, **.lib**, **LEF**), three are constraints on what the result must satisfy (**SDC**, **tech file**, **CTS spec**), and three are state that gets written and re-read across the flow (**DEF**, **ITF/mapping**, **SPEF**). The **netlist** is the gate-level graph: cell instances, nets, and top-level ports. The **.lib** stores cell behavior as **NLDM lookup tables** indexed by input slew and output load, bilinearly interpolated at runtime, and at advanced nodes it stores **CCS** time-varying current waveforms so Miller capacitance and receiver-pin nonlinearity aren’t averaged away. The **LEF** (cell LEF plus tech-LEF) is the physical abstraction so the placer and router never have to open GDS. The **SDC** is the Tcl timing contract: clocks, IO delays, uncertainty, false paths, multicycles. The **technology file** is the foundry rulebook for layer widths, spacings, vias, and multi-patterning colors. The **CTS spec** (CCOPT spec for Innovus) is the declarative description of how the clock network gets built: skew targets, NDR rules, useful-skew limits, and skew groups. The **DEF** is the physical state, emitted at every checkpoint; the DEF grows roughly 30 MB post-floorplan, 80 MB post-place, 120 MB post-CTS once CCOPT inserts roughly 5% more instances as clock buffers, and 350 MB post-route for a 1M-instance block. **ITF** plus the **mapping file** feed the extractor, which emits **SPEF** (one per RC corner) describing every wire’s actual R and C. STA then reads the SPEF, builds Elmore or CCS-aware delays, and reports slack against the SDC. Across MMMC the .lib and SPEF counts multiply: five cell families times three voltages times three temperatures times six corners times four modes lands at over a thousand views, which is why a real signoff run is a coffee break, not a click. The shape of the whole picture: netlist tells you what to build, SDC plus CTS spec tell you how fast and how clean the clock must be, .lib plus LEF plus tech file tell you what is physically and electrically legal, DEF records where everything landed, and ITF plus SPEF say what each routed wire actually does. That is the PD pipeline in ninety seconds.*

Chapter 13

Cumulative Recall Sheet

Every chapter compressed to its Key Idea and mnemonic. One page. Use it as your spaced-repetition surface. If you can read this page and rebuild the rest of the book from it, the book has done its job.

Ch.	Key Idea	Mnemonic
1	PD is not one tool eating one file. It is a pipeline of engines (synthesis hand-off, floorplan, placement, CTS, route, signoff) where each engine demands a specific input no other input can substitute for. Lose one, lose that engine.	NSL-LTD
2	A netlist is a graph of cell instances connected by nets, with top-level ports as the only opening to the outside world. Everything PD does (placement, routing, timing) operates on this graph.	INP
3	A timing library is a characterized lookup of cell behavior across PVT corners. NLDM boils each cell's delay into a 2D table indexed by input slew and output load; the tool bilinearly interpolates between grid points.	Slew in, load out
4	The LEF is the contract between the cell library and the placer/router. It promises footprint, pin layers, and blockages so the tool can resolve placement and routing without ever opening the GDS.	SIZE, SITE, PIN, OBS
5	The SDC is the single source of truth that converts a netlist into a timed promise. Every downstream tool (synthesis, STA, PnR, signoff) measures success against the same clocks, IO delays, uncertainties, and exceptions.	Setup early, hold late
6	The technology file is the foundry rulebook that names every physical layer, fixes its geometric and electrical parameters, and tells the PD tool exactly which shapes it is allowed to draw.	LIVR-C
7	A DEF file is a textual snapshot of the design's physical state at one point in the PD flow. LEF describes cells in the abstract; DEF describes this design's instances in concrete coordinates.	DRC PN
8	The CTS spec is the only place in the flow where you express clock-network intent declaratively; everything else (placement, route, signoff) just reacts to the tree CCOPT builds from it.	Spec skew, shield spine, borrow slack
9	Parasitic files are a three-stage pipeline: ITF describes the process to the extractor, the mapping file binds LEF/DEF layer names to ITF indices, and SPEF records what every wire actually does electrically (R, C, coupling) per RC corner for STA.	ITF cooks, map joins, SPEF tells
10	MMC view count is the product of every characterization axis, not the sum. Adding one new voltage point does not add one view; it multiplies the entire existing matrix by a new factor.	MADG

Chapter 14

Practice Problems

Mixed difficulty. Mixed type. Mixed chapter. The order is **intentionally interleaved**; do not skip around to cluster by topic. Interleaved retrieval beats blocked practice roughly two to one on retention (Rohrer and Taylor).

Problem 1 (Ch. 5 numerical, setup slack)

Given clock period $T = 2$ ns, combinational delay $D = 1.6$ ns, clock skew $= -0.05$ ns, setup time of capture flop $T_s = 0.12$ ns, and clock uncertainty $U = 0.10$ ns. Compute setup slack. Then with $D_{\min} = 0.15$ ns and $T_h = 0.04$ ns, compute hold slack. Which check fails?

Problem 2 (Ch. 1 conceptual, missing input triage)

A PD bring-up script fails on the message `ERROR: cannot resolve cell NAND2_X1`. List the input files where the root cause could live, and for each, state the one debugging command you would run first.

Problem 3 (Ch. 3 numerical, NLDM bilinear interpolation)

Given a 3×3 NLDM `cell_fall` table with input slews $\{30, 80, 200\}$ ps and output loads $\{4, 12, 25\}$ fF, with delay values:

	4 fF	12 fF	25 fF	
30	35	60	95	(ps)
80	55	85	130	
200	110	160	220	

Compute fall delay for slew = 60 ps and load = 18 fF. Show all four corner samples and both interpolation passes.

Problem 4 (Ch. 4 + Ch. 6, grid-and-pitch arithmetic)

A tech-LEF declares M2 with `DIRECTION VERTICAL PITCH 0.14` and track offset 0. A cell-LEF gives PIN A as `RECT 0.10 0.40 0.13 0.80` on M2. Does pin A land on a routing track? If not, what is the smallest shift in the cell's x-origin that fixes it?

Problem 5 (Ch. 7 numerical, DEF coordinate scale)

A DEF header reads `UNITS DISTANCE MICRONS 2000`. A routed wire segment runs from (48000 36000) to (48000 60000) on M3. Give the wire length in microns and the routing direction. Then if the header changes to `UNITS DISTANCE MICRONS 1000`, give the new length.

Problem 6 (Ch. 5 conceptual, SDC misuse)

A junior engineer hits a violating path and adds `set_false_path -from FFA/CK -to FFB/D`. The timing summary turns green. Name three things that can still go wrong in silicon. Name the correct alternative SDC construct for each of: (a) the path is real but only matters every other cycle, (b) the receive flop is on a different clock domain that is truly asynchronous, (c) the launch and capture are on the same clock but the path is by design a half-cycle.

Problem 7 (Ch. 8 numerical, useful skew)

A path runs from FFA to FFB at clock period $T = 1.0$ ns. Pre-CTS, you have setup slack = +50 ps and hold slack = +80 ps on this path. The downstream path FFB to FFC has setup slack = -30 ps (failing) and hold slack = +120 ps. You push useful skew of +40 ps onto FFB's clock. Compute the new setup and hold slacks on BOTH the FFA-to-FFB and FFB-to-FFC paths. Did the trade work?

Problem 8 (Ch. 9 numerical, Elmore delay from SPEF)

A net has a 3-node RC tree: driver $R_d = 50\ \Omega$ to node A, then $R_1 = 20\ \Omega$ from A to sink B with $C_A = 5$ fF at A and $C_B = 8$ fF at B. Compute the Elmore delay at sink B. Then if the SPEF is re-extracted at the C_{worst} corner with all capacitances scaled by 1.25, give the new Elmore delay. Which corner would you use for setup vs hold signoff?

Problem 9 (Ch. 10 conceptual, MMMC explosion)

Your block currently loads 540 .lib views (5 cell families $\times$ 3 voltages $\times$ 3 temperatures $\times$ 6 corners $\times$ 2 modes). Marketing now wants you to support a new low-power mode and a new high-voltage operating point. Without changing anything else, how many .lib views will you load? What's the percentage increase, and which axis dominates the cost?

Problem 10 (Ch. 2 + Ch. 3 + Ch. 5 + Ch. 9 STRETCH)

You inherit a block that closes timing at the typical corner but fails setup at SS by 80 ps on a single endpoint. The endpoint is on a clock-domain crossing into an asynchronous FIFO. Walk through the triage. Specifically: which input file (netlist, .lib, SDC, LEF, SPEF, or CTS spec) would you inspect first, and why? Under what circumstance is the right answer to (a) fix the SDC, (b) re-place the launch and capture flops, (c) request a hold-fix-only ECO at SS, (d) declare a false path, (e) re-extract SPEF at a different RC corner, (f) tighten the CCOPT useful-skew limit on the FIFO clock? For each, name the input file you modify.

Companion Videos

- Ch. 1: <https://youtu.be/fAiTlfTz8KY>
- Ch. 2: <https://youtu.be/5e-o0vP0Y-U>
- Ch. 3: <https://youtu.be/mCaT4w0p4iM>
- Ch. 4: <https://youtu.be/aj0Ys4nQyQY>
- Ch. 5: <https://youtu.be/Lg0mZPUJQ9M>
- Ch. 6: <https://youtu.be/lMfreoc89H0>
- Ch. 7: <https://youtu.be/pnFJ-ADkxX8>